In [166…

```
%pip install scikit-learn
%pip install seaborn
```

```
Requirement already satisfied: scikit-learn in C:\Users\wahee\AppData\Local\Python\p
ythoncore-3.14-64\Lib\site-packages (1.9.1)
Requirement already satisfied: numpy>=1.24.1 in C:\Users\wahee\AppData\Local\Python
\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (2.5.3)
Requirement already satisfied: scipy>=1.10.0 in C:\Users\wahee\AppData\Local\Python
\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (1.18.1)
Requirement already satisfied: joblib>=1.4.0 in C:\Users\wahee\AppData\Local\Python
\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: narwhals>=2.0.1 in C:\Users\wahee\AppData\Local\Pytho
n\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (2.27.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in C:\Users\wahee\AppData\Local
\Python\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in C:\Users\wahee\AppData\Local\Pyth
on\pythoncore-3.14-64\Lib\site-packages (from joblib>=1.4.0->scikit-learn) (3.1.2)
Note: you may need to restart the kernel to use updated packages.
Collecting seaborn
  Using cached seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in C:\Users\wahee\AppData\Local
\Python\pythoncore-3.14-64\Lib\site-packages (from seaborn) (2.5.3)
Requirement already satisfied: pandas>=1.2 in C:\Users\wahee\AppData\Local\Python\py
thoncore-3.14-64\Lib\site-packages (from seaborn) (3.0.6)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in C:\Users\wahee\AppData\Loc
al\Python\pythoncore-3.14-64\Lib\site-packages (from seaborn) (3.11.2)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\wahee\AppData\Local\Pyth
on\pythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.
4.0)
Requirement already satisfied: cycler>=0.10 in C:\Users\wahee\AppData\Local\Python\p
ythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in C:\Users\wahee\AppData\Local\Pyt
hon\pythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.
66.1)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\wahee\AppData\Local\Pyt
hon\pythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.
5.1)
Requirement already satisfied: packaging>=20.0 in C:\Users\wahee\AppData\Local\Pytho
n\pythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.
3)
Requirement already satisfied: pillow>=9 in C:\Users\wahee\AppData\Local\Python\pyth
oncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.3.0)
Requirement already satisfied: pyparsing>=3 in C:\Users\wahee\AppData\Local\Python\p
ythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.3)
Requirement already satisfied: python-dateutil>=2.7 in C:\Users\wahee\AppData\Local
\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\wahee\AppData\Local\Python\pythonc
ore-3.14-64\Lib\site-packages (from pandas>=1.2->seaborn) (2026.4)
Requirement already satisfied: six>=1.5 in C:\Users\wahee\AppData\Local\Python\pytho
ncore-3.14-64\Lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4-
>seaborn) (1.17.0)
Using cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Installing collected packages: seaborn
Successfully installed seaborn-0.13.2
Note: you may need to restart the kernel to use updated packages.
```

In [167…

```
import numpy as np
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers,datasets
from sklearn.model_selection import train_test_split
import random
import pandas as pd
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
```

PART-A

In [90]:
```
import tarfile
import pickle
import numpy as np

x_all, y_all = [], []

with tarfile.open("cifar-10-python.tar.gz", "r:gz") as tar:
    for i in range(1, 6):
        with tar.extractfile(
            f"cifar-10-batches-py/data_batch_{i}"
        ) as f:
            batch = pickle.load(f, encoding="bytes")
            x_all.append(batch[b"data"])
            y_all.extend(batch[b"labels"])

    with tar.extractfile("cifar-10-batches-py/test_batch") as f:
        batch = pickle.load(f, encoding="bytes")
        x_clean = batch[b"data"]
        y_clean = batch[b"labels"]

x_all = np.concatenate(x_all)
y_all = np.array(y_all).reshape(-1, 1)

x_all = x_all.reshape(-1, 3, 32, 32).transpose(0, 2, 3, 1)
x_clean = x_clean.reshape(-1, 3, 32, 32).transpose(0, 2, 3, 1)
y_clean = np.array(y_clean).reshape(-1, 1)

print(x_all.shape, y_all.shape)
print(x_clean.shape, y_clean.shape)
```

```
(50000, 32, 32, 3) (50000, 1)
(10000, 32, 32, 3) (10000, 1)
```

In [91]:
```
SEED = 2026
random.seed(SEED)
np.random.seed(SEED)
tf.keras.utils.set_random_seed(SEED)
```

In [92]:
```
y_all = y_all.flatten()
y_clean = y_clean.flatten()
print("Training Images:", x_all.shape)
print("Training Labels:", y_all.shape)
print("Clean Test Images:", x_clean.shape)
print("Clean Test Labels:", y_clean.shape)
```

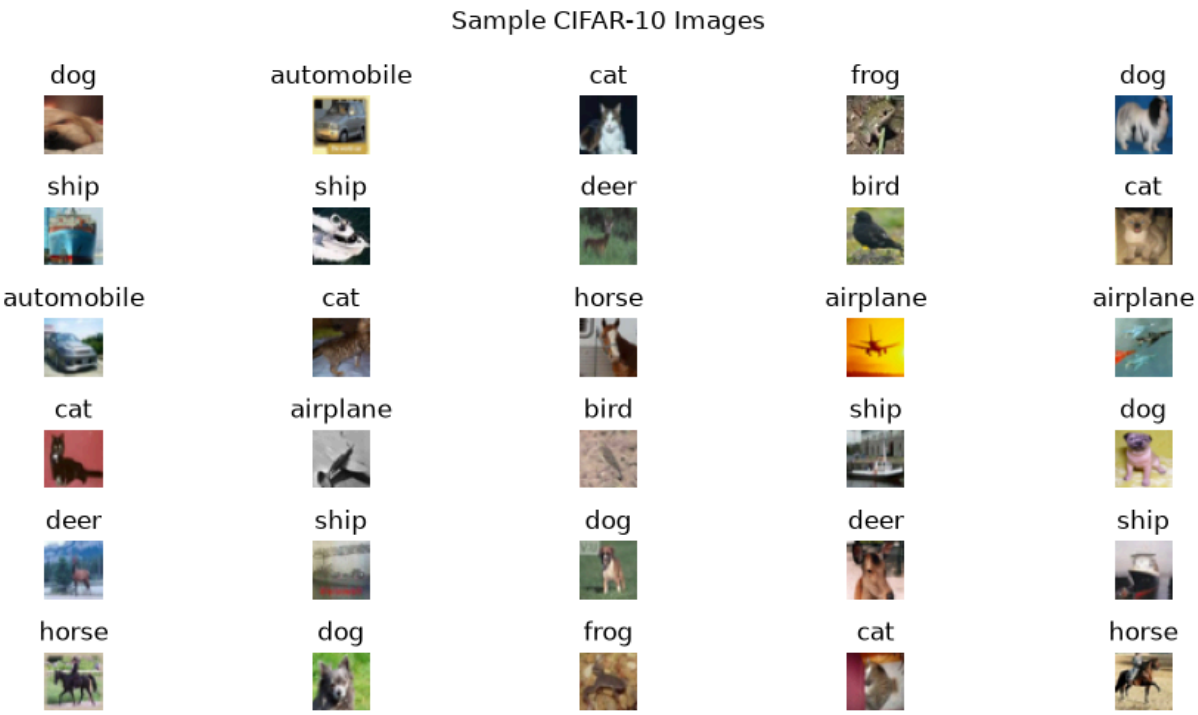
```
Training Images: (50000, 32, 32, 3)
Training Labels: (50000,)
Clean Test Images: (10000, 32, 32, 3)
Clean Test Labels: (10000,)
```

In [93]:
```
x_small, _, y_small, _ = train_test_split(
    x_all,
    y_all,
    train_size=500,
    random_state=SEED,
    stratify=y_all)
print("Small Dataset:", x_small.shape)
print("Labels:", y_small.shape)
```

```
Small Dataset: (500, 32, 32, 3)
Labels: (500,)
```

In [94]:
```
class_names = [
    "airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship"
]
plt.figure(figsize=(10, 5))
for i in range(30):
    plt.subplot(6, 5, i + 1)
    plt.imshow(x_small[i])
    plt.title(class_names[y_small[i]])
    plt.axis("off")

plt.suptitle("Sample CIFAR-10 Images")
plt.tight_layout()
plt.show()
```



In [95]:
```
x_train, x_val, y_train, y_val = train_test_split( x_small,y_small, test_size=100,
    random_state=SEED,
    stratify=y_small
```

```
)
print("Training Images:", x_train.shape)
print("Validation images:", x_val.shape)
```

```
Training Images: (400, 32, 32, 3)
Validation images: (100, 32, 32, 3)
```

In [96]:
```
def inception_module(x, filters=16):

    b1 = layers.Conv2D(
        filters, 1, padding="same", activation="relu"
    )(x)

    b3 = layers.Conv2D(
        filters, 1, padding="same", activation="relu"
    )(x)
    b3 = layers.Conv2D(
        filters, 3, padding="same", activation="relu"
    )(b3)

    b5 = layers.Conv2D(
        filters, 1, padding="same", activation="relu"
    )(x)
    b5 = layers.Conv2D(
        filters, 5, padding="same", activation="relu"
    )(b5)

    bp = layers.AveragePooling2D(
        pool_size=3, strides=1, padding="same"
    )(x)
    bp = layers.Conv2D(
        filters, 1, padding="same", activation="relu"
    )(bp)


    output = layers.Concatenate()([b1, b3, b5, bp])

    return output
```

In [97]:
```
inputs = keras.Input(shape=(32, 32, 3))

x = layers.Conv2D(
    24, 3, padding="same", activation="relu"
)(inputs)

x = layers.MaxPooling2D()(x)

x = layers.Conv2D(
    32, 3, padding="same", activation="relu"
)(x)

x = inception_module(x, filters=16)

x = layers.MaxPooling2D()(x)

x = layers.Conv2D(
```

```
    64, 3, padding="same", activation="relu"
)(x)

x = layers.GlobalAveragePooling2D()(x)



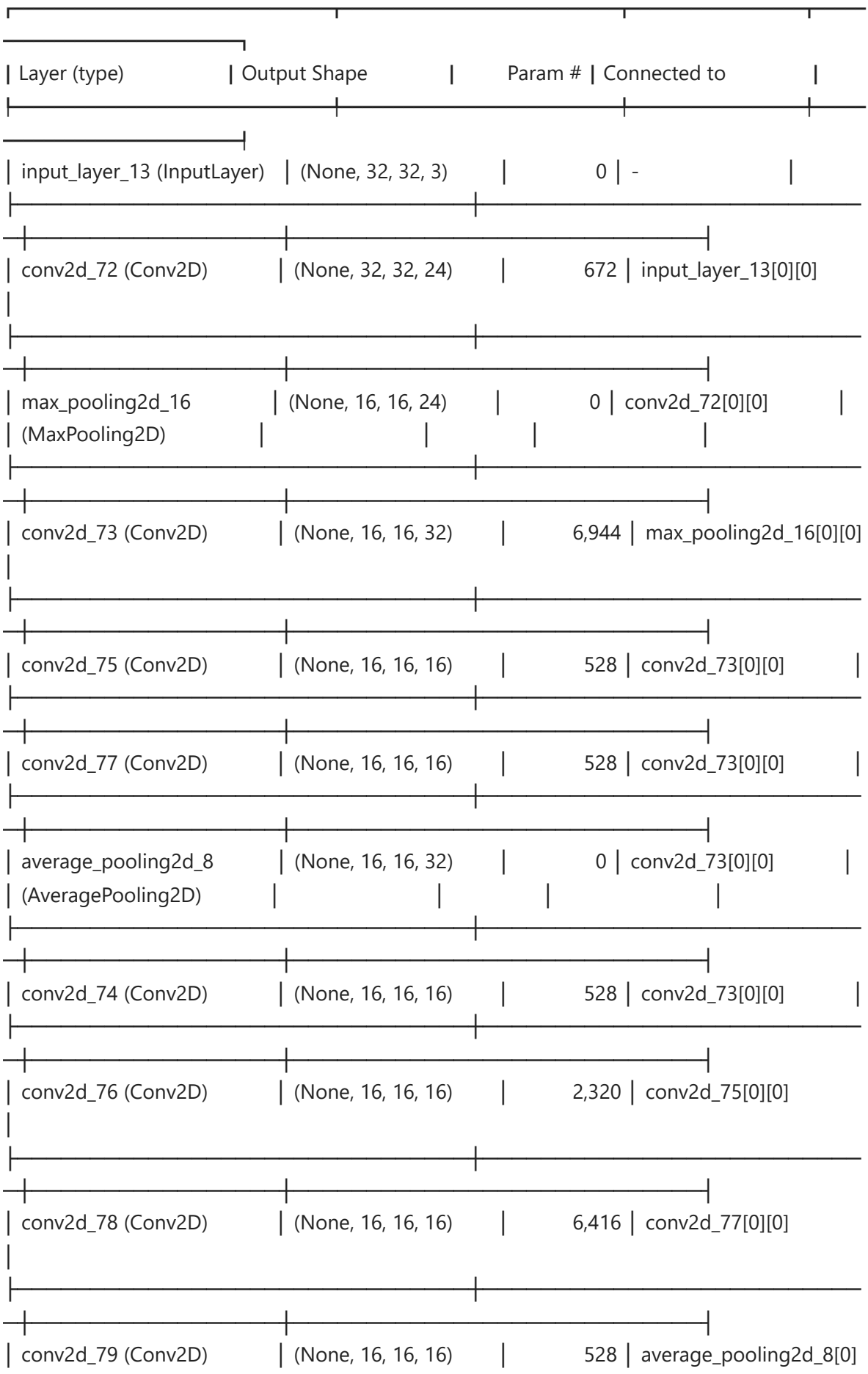
outputs = layers.Dense(
    10, activation="softmax"
)(x)

model = keras.Model(inputs, outputs)

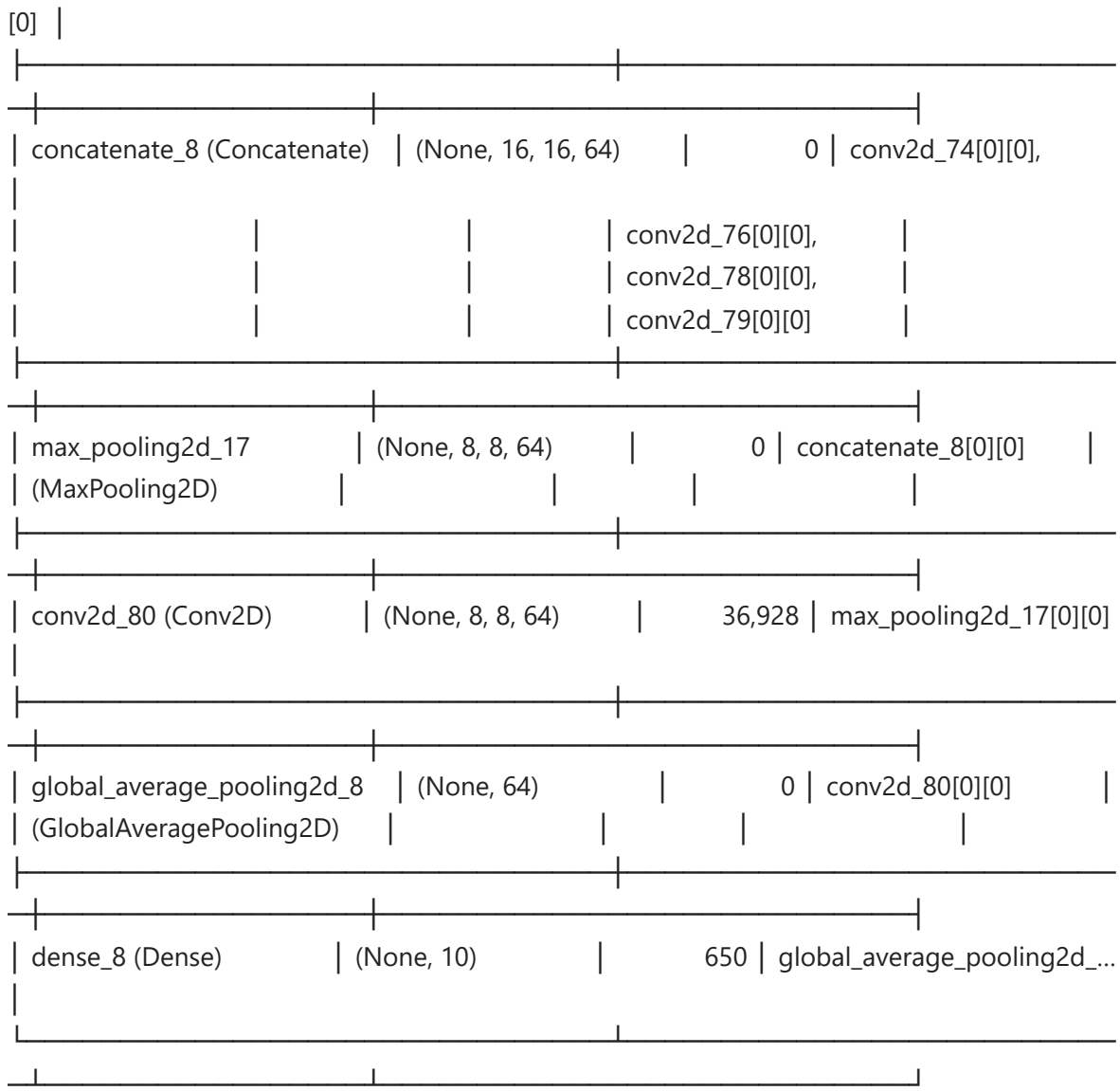
model.summary()

print("Total Parameters:", model.count_params())
```

Model: "functional_13"



Layer (type)	Output Shape	Param #	Connected to
input_layer_13 (InputLayer)	(None, 32, 32, 3)	0	-
conv2d_72 (Conv2D)	(None, 32, 32, 24)	672	input_layer_13[0][0]
max_pooling2d_16 (MaxPooling2D)	(None, 16, 16, 24)	0	conv2d_72[0][0]
conv2d_73 (Conv2D)	(None, 16, 16, 32)	6,944	max_pooling2d_16[0][0]
conv2d_75 (Conv2D)	(None, 16, 16, 16)	528	conv2d_73[0][0]
conv2d_77 (Conv2D)	(None, 16, 16, 16)	528	conv2d_73[0][0]
average_pooling2d_8 (AveragePooling2D)	(None, 16, 16, 32)	0	conv2d_73[0][0]
conv2d_74 (Conv2D)	(None, 16, 16, 16)	528	conv2d_73[0][0]
conv2d_76 (Conv2D)	(None, 16, 16, 16)	2,320	conv2d_75[0][0]
conv2d_78 (Conv2D)	(None, 16, 16, 16)	6,416	conv2d_77[0][0]
conv2d_79 (Conv2D)	(None, 16, 16, 16)	528	average_pooling2d_8[0]



Total params: 56,042 (218.91 KB)

Trainable params: 56,042 (218.91 KB)

Non-trainable params: 0 (0.00 B)

```
Total Parameters: 56042
```

In [98]:
```
model.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

In [99]:
```
history = model.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
```

```
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 4s 163ms/step - accuracy: 0.0950 - loss: 13.5968 - val_ac
curacy: 0.1600 - val_loss: 4.4655
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.1175 - loss: 3.3589 - val_accu
racy: 0.1100 - val_loss: 2.5367
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.1400 - loss: 2.3505 - val_accu
racy: 0.1700 - val_loss: 2.2297
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.1850 - loss: 2.2255 - val_accu
racy: 0.2600 - val_loss: 2.1176
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2225 - loss: 2.1222 - val_accu
racy: 0.2800 - val_loss: 2.0143
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.2775 - loss: 2.0355 - val_accu
racy: 0.2900 - val_loss: 1.9214
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.2925 - loss: 1.9582 - val_accu
racy: 0.3200 - val_loss: 1.8951
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.3200 - loss: 1.9116 - val_accu
racy: 0.2900 - val_loss: 1.8446
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.3650 - loss: 1.8523 - val_accu
racy: 0.3000 - val_loss: 1.8062
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.3775 - loss: 1.7848 - val_accu
racy: 0.3400 - val_loss: 1.7808
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.3925 - loss: 1.7218 - val_accu
racy: 0.3100 - val_loss: 1.7572
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.4200 - loss: 1.6789 - val_accu
racy: 0.3200 - val_loss: 1.7213
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.4450 - loss: 1.6255 - val_accu
racy: 0.3300 - val_loss: 1.7118
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.4575 - loss: 1.5826 - val_accu
racy: 0.3600 - val_loss: 1.7018
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.4650 - loss: 1.5382 - val_accu
racy: 0.3700 - val_loss: 1.6935
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.4800 - loss: 1.4963 - val_accu
racy: 0.3800 - val_loss: 1.6928
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.5050 - loss: 1.4553 - val_accu
racy: 0.3700 - val_loss: 1.7027
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.5225 - loss: 1.4163 - val_accu
racy: 0.3800 - val_loss: 1.7065
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.5400 - loss: 1.3758 - val_accu
```

```
racy: 0.3600 - val_loss: 1.7249
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.5575 - loss: 1.3327 - val_accu
racy: 0.3700 - val_loss: 1.7465
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.5800 - loss: 1.2903 - val_accu
racy: 0.3700 - val_loss: 1.7543
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.5850 - loss: 1.2454 - val_accu
racy: 0.3800 - val_loss: 1.7844
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.6200 - loss: 1.1954 - val_accu
racy: 0.3400 - val_loss: 1.8188
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.6300 - loss: 1.1575 - val_accu
racy: 0.3200 - val_loss: 1.8778
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.6100 - loss: 1.1558 - val_accu
racy: 0.2800 - val_loss: 1.9477
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.6000 - loss: 1.1627 - val_accu
racy: 0.3400 - val_loss: 1.8550
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.6000 - loss: 1.1387 - val_accu
racy: 0.3700 - val_loss: 1.8094
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.6000 - loss: 1.1237 - val_accu
racy: 0.3200 - val_loss: 2.0801
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.6450 - loss: 1.0713 - val_accu
racy: 0.3800 - val_loss: 2.0627
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.6525 - loss: 1.0365 - val_accu
racy: 0.3200 - val_loss: 2.0350
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6825 - loss: 0.9841 - val_accu
racy: 0.3500 - val_loss: 1.9291
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.7250 - loss: 0.8696 - val_accu
racy: 0.3200 - val_loss: 1.8553
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.7475 - loss: 0.8289 - val_accu
racy: 0.3200 - val_loss: 2.0066
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.7375 - loss: 0.7883 - val_accu
racy: 0.3300 - val_loss: 1.9427
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.7625 - loss: 0.7432 - val_accu
racy: 0.3600 - val_loss: 2.0151
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8125 - loss: 0.6637 - val_accu
racy: 0.3500 - val_loss: 1.9459
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8000 - loss: 0.6189 - val_accu
racy: 0.3800 - val_loss: 2.0220
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8275 - loss: 0.5957 - val_accu
racy: 0.3700 - val_loss: 2.0655
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8525 - loss: 0.5509 - val_accu
racy: 0.3400 - val_loss: 2.2270
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8175 - loss: 0.5861 - val_accu
racy: 0.3400 - val_loss: 2.3544
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8100 - loss: 0.6014 - val_accu
racy: 0.3200 - val_loss: 2.5532
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.7925 - loss: 0.6441 - val_accu
racy: 0.3800 - val_loss: 2.1455
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8075 - loss: 0.6310 - val_accu
racy: 0.3800 - val_loss: 2.0179
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8250 - loss: 0.5715 - val_accu
racy: 0.3800 - val_loss: 2.2697
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.8550 - loss: 0.4749 - val_accu
racy: 0.3900 - val_loss: 2.2570
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9000 - loss: 0.3942 - val_accu
racy: 0.3100 - val_loss: 2.5761
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8575 - loss: 0.4192 - val_accu
racy: 0.3800 - val_loss: 2.3713
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8500 - loss: 0.4675 - val_accu
racy: 0.3300 - val_loss: 2.6619
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8100 - loss: 0.5498 - val_accu
racy: 0.3100 - val_loss: 2.7832
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8400 - loss: 0.5170 - val_accu
racy: 0.3900 - val_loss: 2.3551
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8325 - loss: 0.4656 - val_accu
racy: 0.3600 - val_loss: 2.7664
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8200 - loss: 0.5054 - val_accu
racy: 0.4000 - val_loss: 2.1595
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8375 - loss: 0.4635 - val_accu
racy: 0.3500 - val_loss: 2.4270
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8725 - loss: 0.4248 - val_accu
racy: 0.3800 - val_loss: 2.4638
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.9025 - loss: 0.3345 - val_accu
racy: 0.4100 - val_loss: 2.7053
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8575 - loss: 0.4121 - val_accu
racy: 0.3600 - val_loss: 2.5432
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8550 - loss: 0.4175 - val_accu
racy: 0.3600 - val_loss: 3.0234
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8300 - loss: 0.4861 - val_accu
racy: 0.3200 - val_loss: 3.1142
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8025 - loss: 0.6156 - val_accu
racy: 0.3000 - val_loss: 3.1173
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8350 - loss: 0.4588 - val_accu
racy: 0.3500 - val_loss: 3.1671
```

In [100…

```
train_loss, train_acc = model.evaluate(
    x_train, y_train, verbose=0
)

val_loss, val_acc = model.evaluate(
    x_val, y_val, verbose=0
)

print("Training Accuracy:", train_acc)
print("Validation Accuracy:", val_acc)
```

```
Training Accuracy: 0.8374999761581421
Validation Accuracy: 0.3499999940395355
```

In [101…

```
clean_loss, clean_acc = model.evaluate(
    x_clean, y_clean, verbose=1
)

print("Clean Test Accuracy:", clean_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.3174 - loss: 3.3606
Clean Test Accuracy: 0.3174000084400177
```

In [102…

```
print("Clean images:", x_clean.shape)
print("Clean labels:", y_clean.shape)
print("Image pixel range:", x_clean.min(), x_clean.max())
```

```
Clean images: (10000, 32, 32, 3)
Clean labels: (10000,)
Image pixel range: 0 255
```

In [103…

```
x_shifted = x_clean.astype("float32").copy()

rotation = tf.keras.layers.RandomRotation(
    factor=25 / 360,
    fill_mode="nearest",
    seed=SEED
)

brightness = tf.keras.layers.RandomBrightness(
    factor=0.2,
    value_range=(0, 255),
    seed=SEED
)
```

```
contrast = tf.keras.layers.RandomContrast(
    factor=0.2,
    seed=SEED
)

x_shifted = rotation(x_shifted, training=True)
x_shifted = brightness(x_shifted, training=True)
x_shifted = contrast(x_shifted, training=True)

x_shifted = tf.clip_by_value(
    x_shifted, 0, 255
).numpy()

print("Shifted images:", x_shifted.shape)
print("Pixel range:", x_shifted.min(), x_shifted.max())
```

```
Shifted images: (10000, 32, 32, 3)
Pixel range: 0.0 255.0
```

In [104…

```
rng = np.random.default_rng(SEED)

for i in range(len(x_shifted)):
    h, w, _ = x_shifted[i].shape

    box_h = h // 2
    box_w = w // 2

    top = rng.integers(0, h - box_h + 1)
    left = rng.integers(0, w - box_w + 1)

    x_shifted[
        i,
        top:top + box_h,
        left:left + box_w,
        :
    ] = 0

print("occlusion successfully")
```

```
occlusion successfully
```

In [105…

```
y_shifted = y_clean.copy()

print("Shifted labels:", y_shifted.shape)
```

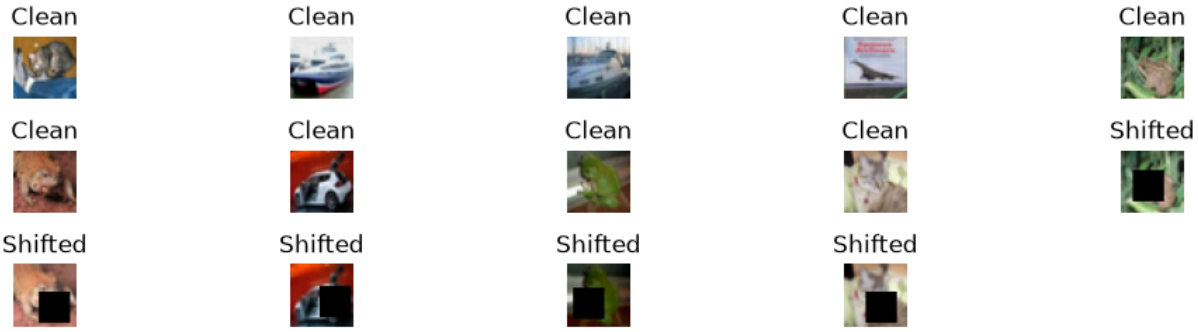
```
Shifted labels: (10000,)
```

In [106…

```
class_names = [
    "airplane", "automobile", "bird", "cat", "deer","dog", "frog", "horse", "ship",
]
plt.figure(figsize=(10, 5))
for i in range(9):
    plt.subplot(6, 5, i + 1)
    plt.imshow(x_clean[i].astype("uint8"))
    plt.title("Clean")
    plt.axis("off")
```

```
    plt.subplot(6, 5, i + 6)
    plt.imshow(np.clip(x_shifted[i], 0, 255).astype("uint8"))
    plt.title("Shifted")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



In [107…

```
shifted_loss, shifted_acc = model.evaluate(
    x_shifted,
    y_shifted,
    verbose=1
)

print("Shifted Test Loss:", shifted_loss)
print("Shifted Test Accuracy:", shifted_acc * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.1577 - loss: 8.5502
Shifted Test Loss: 8.550195693969727
Shifted Test Accuracy: 15.770000219345093 %
```

In [108…

```
x_shifted = x_shifted.astype("float32")

shifted_loss, shifted_acc = model.evaluate(
    x_shifted, y_shifted, verbose=1
)

print("Shifted Test Accuracy:", shifted_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.1577 - loss: 8.5502
Shifted Test Accuracy: 0.15770000219345093
```

In [109…

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("Baseline Accuracy")
```

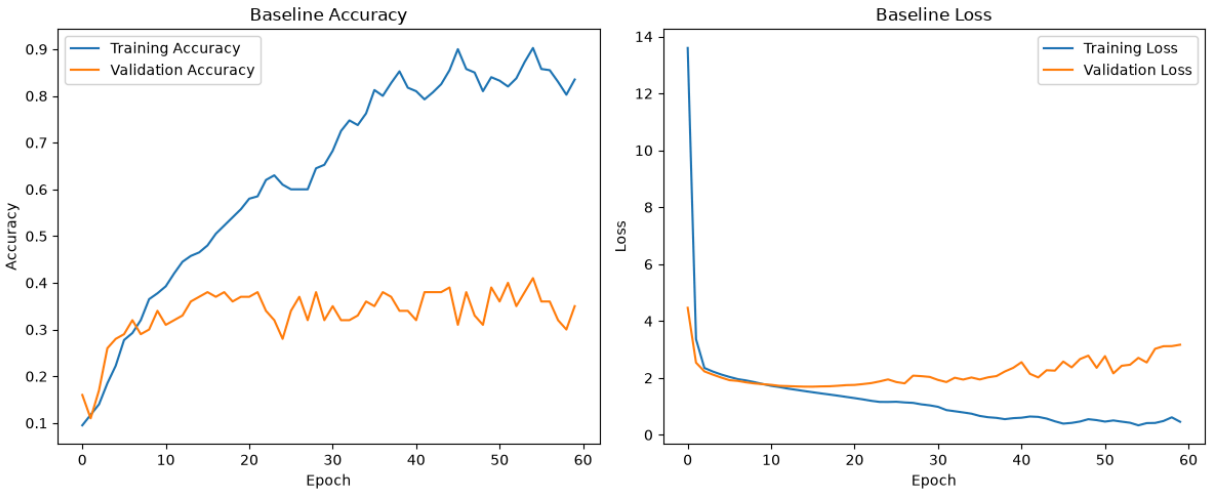
```
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()


plt.subplot(1, 2, 2)

plt.plot(
    history.history["loss"],
    label="Training Loss"
)
plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Baseline Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()
```



SEED 2026

In [110…

```
results = {
    "Dataset": [
        "Training",
        "Validation",
        "Clean Test",
        "Shifted Test"
    ],
    "Accuracy (%)": [
        train_acc * 100,
        val_acc * 100,
        clean_acc * 100,
        shifted_acc * 100
    ]
}
```

```
results_df = pd.DataFrame(results)

results_df["Accuracy (%)"] = results_df["Accuracy (%)"].round(2)

display(results_df)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Dataset</th> <th>Accuracy (%)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> </tbody> <tbody></tbody>

	Dataset	Accuracy (%)
0	Training	83.75
1	Validation	35.00
2	Clean Test	31.74
3	Shifted Test	15.77

The model learns from the training images. If training accuracy is much higher than validation accuracy it may be overfitting. Shifted images are harder to predict because they have changes like rotation, brightness, and noise. So, the model struggles with new image changes.

PART B

In [111…

```
tf.keras.utils.set_random_seed(SEED)

data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),

    layers.RandomTranslation(
        height_factor=0.1,
        width_factor=0.1,
        fill_mode="nearest"
    ),

    layers.RandomRotation(
        factor=25 / 360,
        fill_mode="nearest"
    ),

    layers.RandomBrightness(
        factor=0.2,
        value_range=(0, 255)
    ),

    layers.RandomContrast(factor=0.2)
], name="data_augmentation")
```

In [112…

```
inputs = keras.Input(shape=(32, 32, 3))

x = data_augmentation(inputs)
```

```
x = layers.Conv2D(
    24, 3, padding="same", activation="relu"
)(x)

x = layers.MaxPooling2D()(x)

x = layers.Conv2D(
    32, 3, padding="same", activation="relu"
)(x)

x = inception_module(x, filters=16)

x = layers.MaxPooling2D()(x)

x = layers.Conv2D(
    64, 3, padding="same", activation="relu"
)(x)

x = layers.GlobalAveragePooling2D()(x)

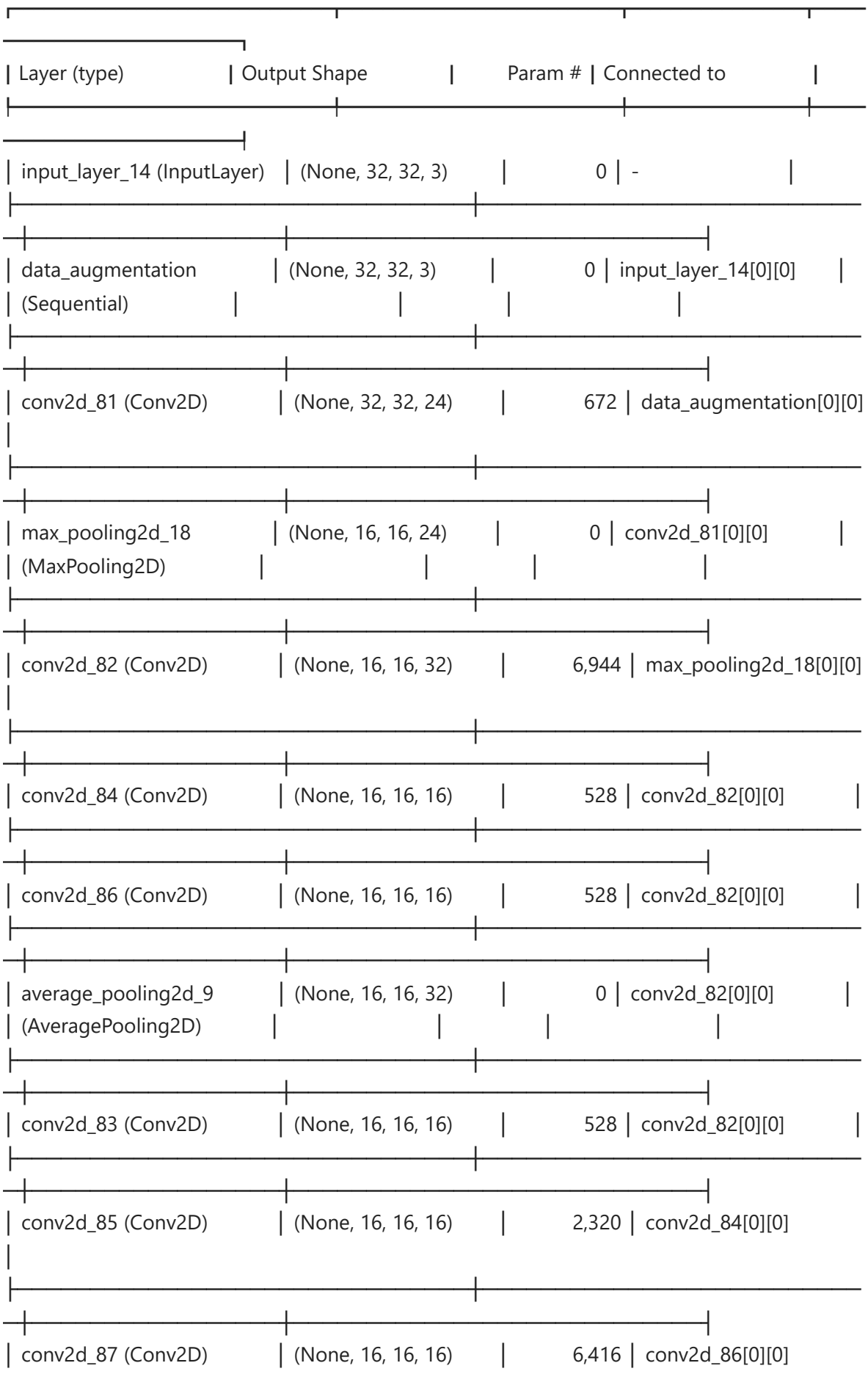
outputs = layers.Dense(
    10, activation="softmax"
)(x)

model_aug = keras.Model(inputs, outputs)

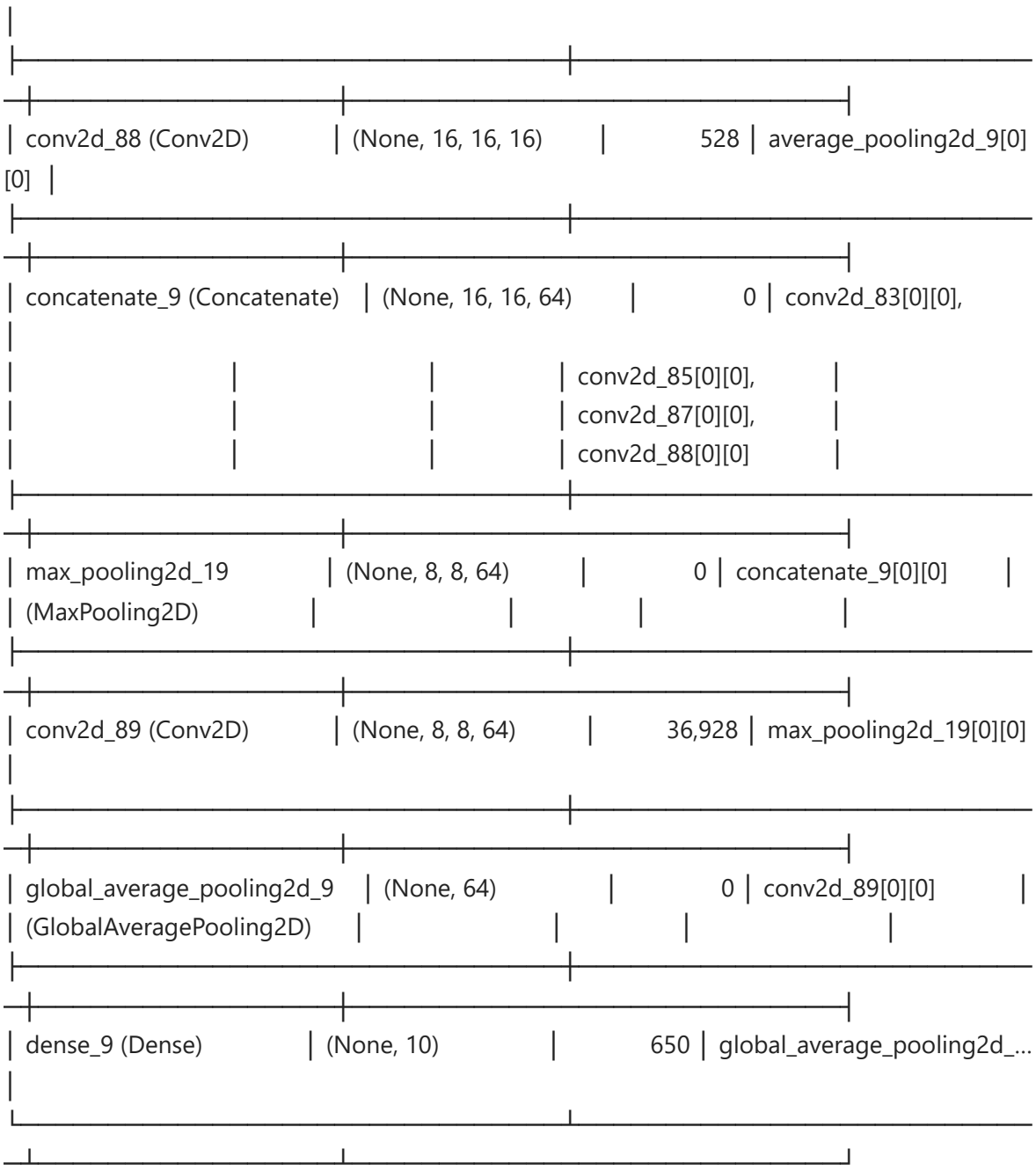
model_aug.summary()

print("Trainable parameters:",
      sum(np.prod(w.shape) for w in model_aug.trainable_weights))
```

Model: "functional_15"



Layer (type)	Output Shape	Param #	Connected to
input_layer_14 (InputLayer)	(None, 32, 32, 3)	0	-
data_augmentation (Sequential)	(None, 32, 32, 3)	0	input_layer_14[0][0]
conv2d_81 (Conv2D)	(None, 32, 32, 24)	672	data_augmentation[0][0]
max_pooling2d_18 (MaxPooling2D)	(None, 16, 16, 24)	0	conv2d_81[0][0]
conv2d_82 (Conv2D)	(None, 16, 16, 32)	6,944	max_pooling2d_18[0][0]
conv2d_84 (Conv2D)	(None, 16, 16, 16)	528	conv2d_82[0][0]
conv2d_86 (Conv2D)	(None, 16, 16, 16)	528	conv2d_82[0][0]
average_pooling2d_9 (AveragePooling2D)	(None, 16, 16, 32)	0	conv2d_82[0][0]
conv2d_83 (Conv2D)	(None, 16, 16, 16)	528	conv2d_82[0][0]
conv2d_85 (Conv2D)	(None, 16, 16, 16)	2,320	conv2d_84[0][0]
conv2d_87 (Conv2D)	(None, 16, 16, 16)	6,416	conv2d_86[0][0]



Total params: 56,042 (218.91 KB)

Trainable params: 56,042 (218.91 KB)

Non-trainable params: 0 (0.00 B)

```
Trainable parameters: 56042
```

In [113…

```
model_aug.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

In [114…

```
history_aug = model_aug.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 190ms/step - accuracy: 0.0775 - loss: 6.3949 - val_acc
uracy: 0.1000 - val_loss: 2.5362
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.1125 - loss: 2.3956 - val_accu
racy: 0.1200 - val_loss: 2.3147
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.1375 - loss: 2.2881 - val_accu
racy: 0.1700 - val_loss: 2.2144
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.1525 - loss: 2.2540 - val_accu
racy: 0.2400 - val_loss: 2.1538
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.2000 - loss: 2.1724 - val_accu
racy: 0.2200 - val_loss: 2.1062
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.2025 - loss: 2.1059 - val_accu
racy: 0.2300 - val_loss: 2.0703
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.2425 - loss: 2.0765 - val_accu
racy: 0.2100 - val_loss: 2.0702
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2625 - loss: 2.0187 - val_accu
racy: 0.2600 - val_loss: 1.9838
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.2600 - loss: 1.9834 - val_accu
racy: 0.3000 - val_loss: 1.9042
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2775 - loss: 1.9617 - val_accu
racy: 0.2700 - val_loss: 1.9026
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.2750 - loss: 1.9630 - val_accu
racy: 0.3200 - val_loss: 1.8713
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.2925 - loss: 1.9165 - val_accu
racy: 0.3400 - val_loss: 1.8557
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.3200 - loss: 1.8495 - val_accu
racy: 0.3500 - val_loss: 1.7704
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.3375 - loss: 1.7913 - val_accu
racy: 0.3300 - val_loss: 1.7407
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.3600 - loss: 1.7467 - val_accu
racy: 0.3200 - val_loss: 1.7324
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.3625 - loss: 1.7530 - val_accu
racy: 0.2900 - val_loss: 1.7365
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.3725 - loss: 1.7556 - val_accu
racy: 0.3500 - val_loss: 1.6995
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.3875 - loss: 1.6969 - val_accu
racy: 0.3400 - val_loss: 1.6950
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.3425 - loss: 1.7316 - val_accu
```

```
racy: 0.3500 - val_loss: 1.7297
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.3550 - loss: 1.7337 - val_accu
racy: 0.3600 - val_loss: 1.6919
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3750 - loss: 1.7151 - val_accu
racy: 0.3900 - val_loss: 1.7264
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - accuracy: 0.3775 - loss: 1.6513 - val_accu
racy: 0.3800 - val_loss: 1.6130
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.3875 - loss: 1.7160 - val_accu
racy: 0.4300 - val_loss: 1.6363
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.3975 - loss: 1.5891 - val_accu
racy: 0.3400 - val_loss: 1.7171
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.4650 - loss: 1.5633 - val_accu
racy: 0.4300 - val_loss: 1.6576
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.4475 - loss: 1.5291 - val_accu
racy: 0.4200 - val_loss: 1.6913
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.4200 - loss: 1.5889 - val_accu
racy: 0.3800 - val_loss: 1.6311
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.4100 - loss: 1.5394 - val_accu
racy: 0.3800 - val_loss: 1.6880
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.4700 - loss: 1.4919 - val_accu
racy: 0.3700 - val_loss: 1.6953
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.4500 - loss: 1.5107 - val_accu
racy: 0.3800 - val_loss: 1.6943
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.4725 - loss: 1.5046 - val_accu
racy: 0.3900 - val_loss: 1.6348
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.4650 - loss: 1.4859 - val_accu
racy: 0.4300 - val_loss: 1.6088
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.4525 - loss: 1.4459 - val_accu
racy: 0.3500 - val_loss: 1.8081
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - accuracy: 0.4675 - loss: 1.4995 - val_accu
racy: 0.4400 - val_loss: 1.6374
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.4750 - loss: 1.4320 - val_accu
racy: 0.3700 - val_loss: 1.7240
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.4750 - loss: 1.4184 - val_accu
racy: 0.3600 - val_loss: 1.7706
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.5075 - loss: 1.4131 - val_accu
racy: 0.3600 - val_loss: 1.7027
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.4775 - loss: 1.3996 - val_accu
racy: 0.3800 - val_loss: 1.7002
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.5075 - loss: 1.4057 - val_accu
racy: 0.3600 - val_loss: 1.7407
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.5325 - loss: 1.3148 - val_accu
racy: 0.3700 - val_loss: 1.6550
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.5050 - loss: 1.3455 - val_accu
racy: 0.3100 - val_loss: 1.9687
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.4950 - loss: 1.3378 - val_accu
racy: 0.3800 - val_loss: 1.7100
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.5175 - loss: 1.3639 - val_accu
racy: 0.3400 - val_loss: 1.7167
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.5325 - loss: 1.3445 - val_accu
racy: 0.3700 - val_loss: 1.6629
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.5375 - loss: 1.3731 - val_accu
racy: 0.3900 - val_loss: 1.8149
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.5175 - loss: 1.3029 - val_accu
racy: 0.3500 - val_loss: 1.9125
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.5400 - loss: 1.2866 - val_accu
racy: 0.3600 - val_loss: 1.7292
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.5350 - loss: 1.2513 - val_accu
racy: 0.3100 - val_loss: 1.9807
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.5825 - loss: 1.2499 - val_accu
racy: 0.4100 - val_loss: 1.7084
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.5475 - loss: 1.2616 - val_accu
racy: 0.3800 - val_loss: 1.8969
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.5700 - loss: 1.2391 - val_accu
racy: 0.3600 - val_loss: 1.7576
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.5175 - loss: 1.2604 - val_accu
racy: 0.3400 - val_loss: 1.9356
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.5475 - loss: 1.2738 - val_accu
racy: 0.3300 - val_loss: 2.0240
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.5400 - loss: 1.3205 - val_accu
racy: 0.3400 - val_loss: 1.7781
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.5600 - loss: 1.2468 - val_accu
racy: 0.3400 - val_loss: 2.0383
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.5775 - loss: 1.2113 - val_accu
racy: 0.3200 - val_loss: 1.9694
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.5650 - loss: 1.2298 - val_accu
racy: 0.3300 - val_loss: 1.6704
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.5550 - loss: 1.2207 - val_accu
racy: 0.2900 - val_loss: 2.1551
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.5650 - loss: 1.2015 - val_accu
racy: 0.3100 - val_loss: 1.9041
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.5700 - loss: 1.1859 - val_accu
racy: 0.3600 - val_loss: 2.0092
```

In [115…

```python
train_loss_aug, train_acc_aug = model_aug.evaluate(
    x_train, y_train, verbose=0
)

val_loss_aug, val_acc_aug = model_aug.evaluate(
    x_val, y_val, verbose=0
)

clean_loss_aug, clean_acc_aug = model_aug.evaluate(
    x_clean, y_clean, verbose=1
)

print("Training Accuracy:", train_acc_aug * 100, "%")
print("Validation Accuracy:", val_acc_aug * 100, "%")
print("Clean Test Accuracy:", clean_acc_aug * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.3418 - loss: 2.2341
Training Accuracy: 55.50000071525574 %
Validation Accuracy: 36.000001430511475 %
Clean Test Accuracy: 34.18000042438507 %
```

In [116…

```python
shifted_loss_aug, shifted_acc_aug = model_aug.evaluate(
    x_shifted, y_shifted, verbose=1
)

print("Synthetic Shifted Test Accuracy:",
      shifted_acc_aug * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.2273 - loss: 2.8835
Synthetic Shifted Test Accuracy: 22.73000031709671 %
```

In [117…

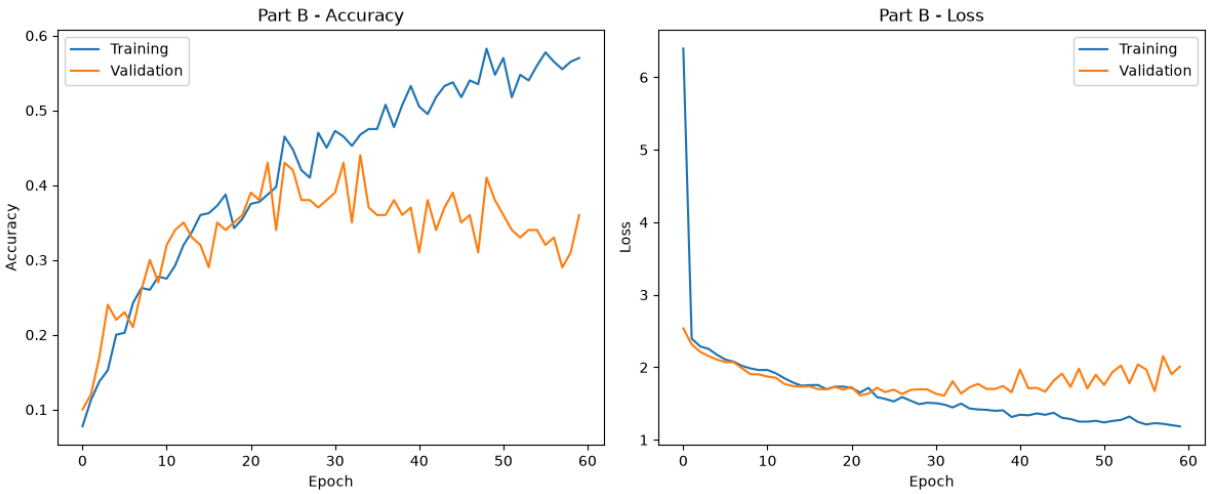
```python
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_aug.history["accuracy"],
         label="Training")
plt.plot(history_aug.history["val_accuracy"],
         label="Validation")
plt.title("Part B - Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
```

```
plt.plot(history_aug.history["loss"],
         label="Training")
plt.plot(history_aug.history["val_loss"],
         label="Validation")
plt.title("Part B - Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()
```



<h2 id="Would-vertical-flip-help-or-hurt-on-this-dataset?-Why?">Would vertical flip help or hurt on this dataset? Why?</h2> Vertical flip may hurt accuracy because most of theobjects normally appear upright.Horizontal flip is more useful because it changes the left right direction without changing the objects class. <h2 id="If-you-augment-with-rotation-up-to-%C2%B190%C2%B0-but-the-test-only-has-%C2%B125%C2%B0,-what-is-the-trade-off?">If you augment with rotation up to ±90° but the test only has ±25°, what is the trade off?</h2> Large rotations may Improve performance on rotated imagebut they can also create unusual images and make training harder. Since test images rotate only up to ±25 smaller rotations are more suitable. <h2 id="Why-can-too-much-augmentation-reduce-accuracy-on-such-a-small-model?">Why can too much augmentation reduce accuracy on such a small model?</h2> Too much augmentation can make images too difficult or unrealistic. A small model trained on only 400 images may struggle to learn useful features. Therefore, we should use suitable augmentation and check its performance on validation data.

PART C

In [118…

```
def conv_bn_relu(x, filters, kernel_size):
    x = layers.Conv2D(
        filters,
        kernel_size,
        padding="same",
        use_bias=False
    )(x)
```

```
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    return x


def inception_c(x, filters=16):

    b1 = conv_bn_relu(x, filters, 1)

    b3 = conv_bn_relu(x, filters, 1)
    b3 = conv_bn_relu(b3, filters, 3)

    b5 = conv_bn_relu(x, filters, 1)
    b5 = conv_bn_relu(b5, filters, 5)

    bp = layers.AveragePooling2D(
        pool_size=3, strides=1, padding="same"
    )(x)
    bp = conv_bn_relu(bp, filters, 1)

    return layers.Concatenate()([b1, b3, b5, bp])


inputs = keras.Input(shape=(32, 32, 3))

x = data_augmentation(inputs)

x = conv_bn_relu(x, 24, 3)
x = layers.MaxPooling2D()(x)

x = conv_bn_relu(x, 32, 3)
x = inception_c(x, filters=16)

x = layers.MaxPooling2D()(x)
x = conv_bn_relu(x, 64, 3)

x = layers.GlobalAveragePooling2D()(x)

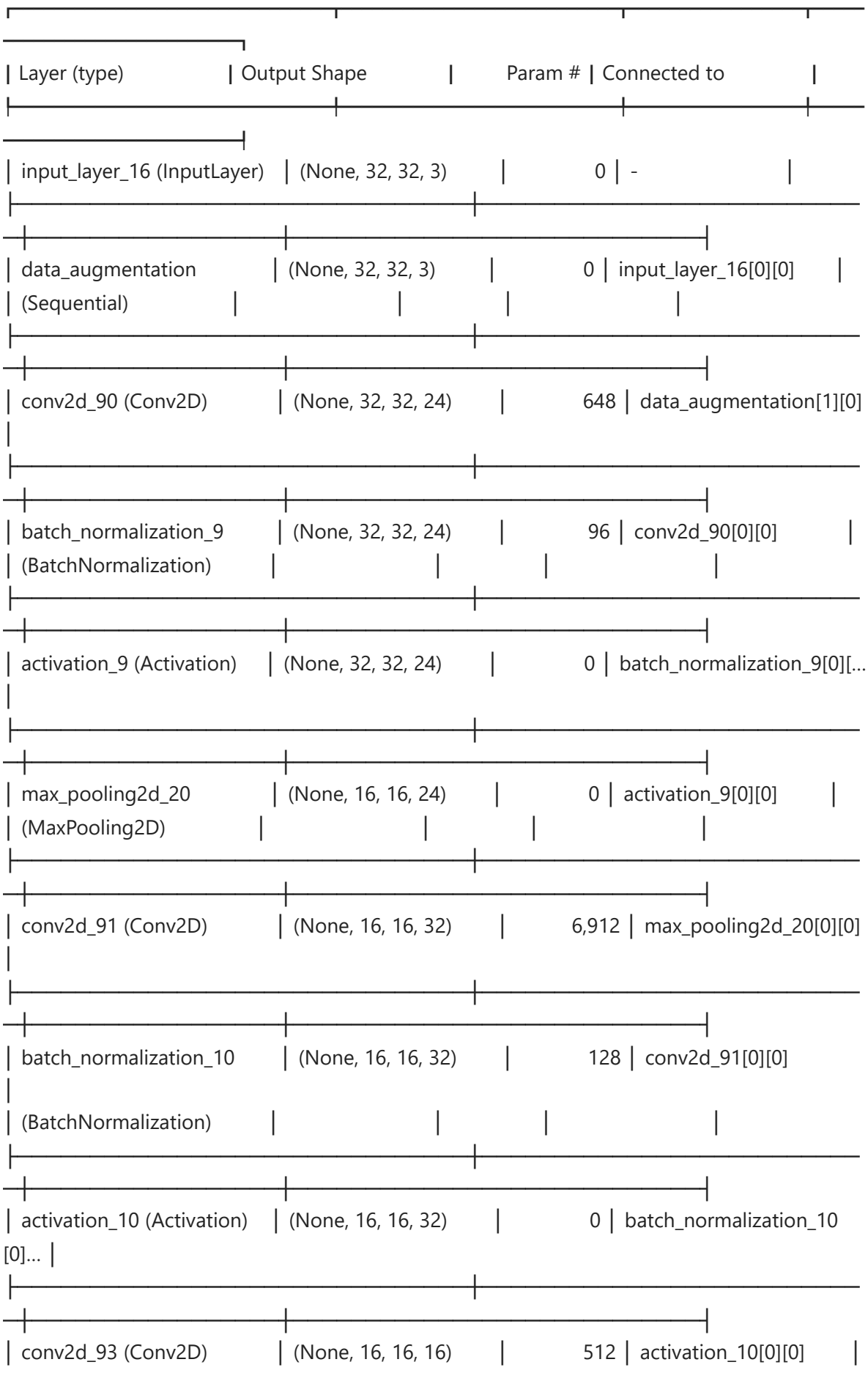
# Dropout
x = layers.Dropout(0.4)(x)

outputs = layers.Dense(10, activation="softmax")(x)

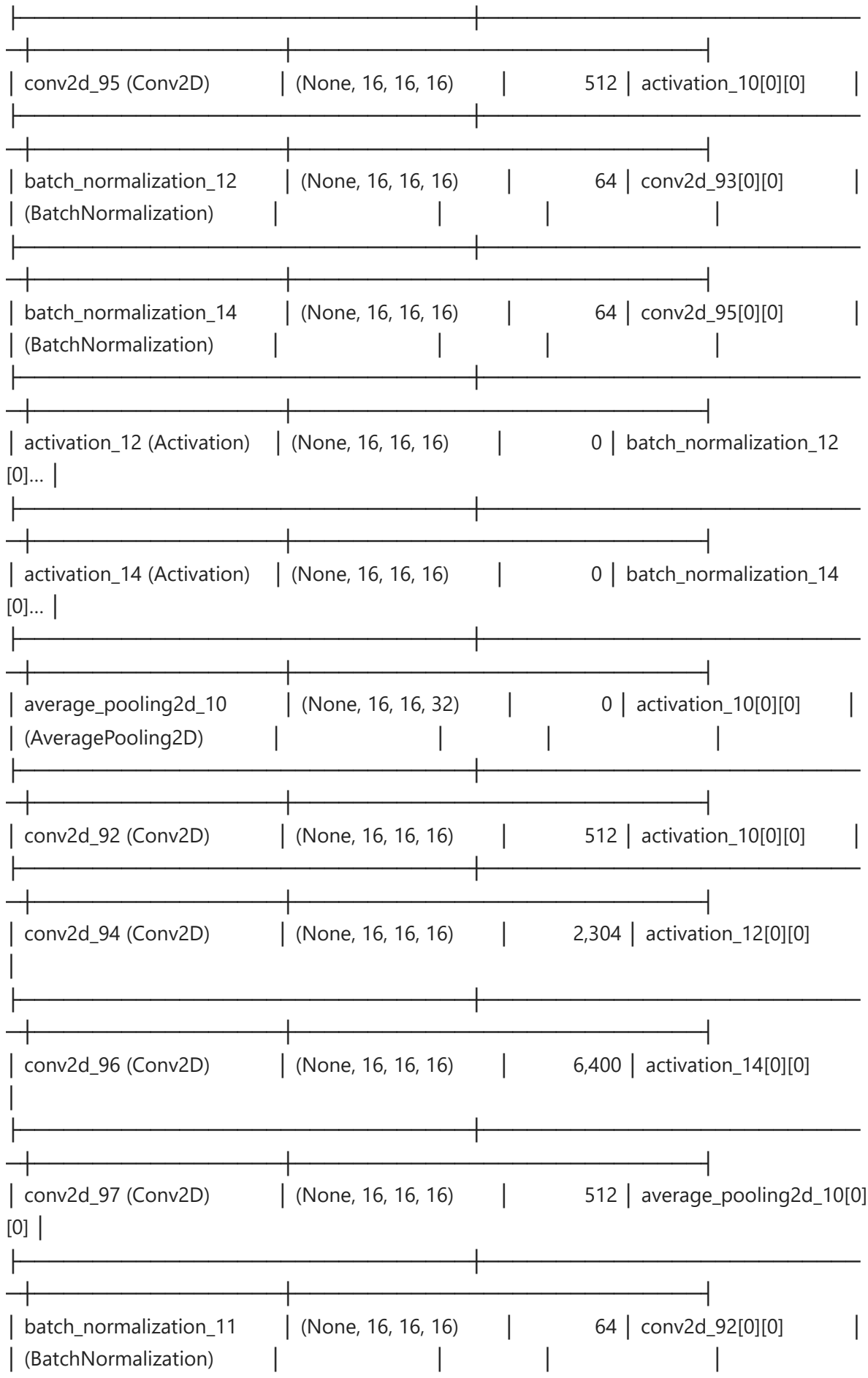
model_c = keras.Model(inputs, outputs)

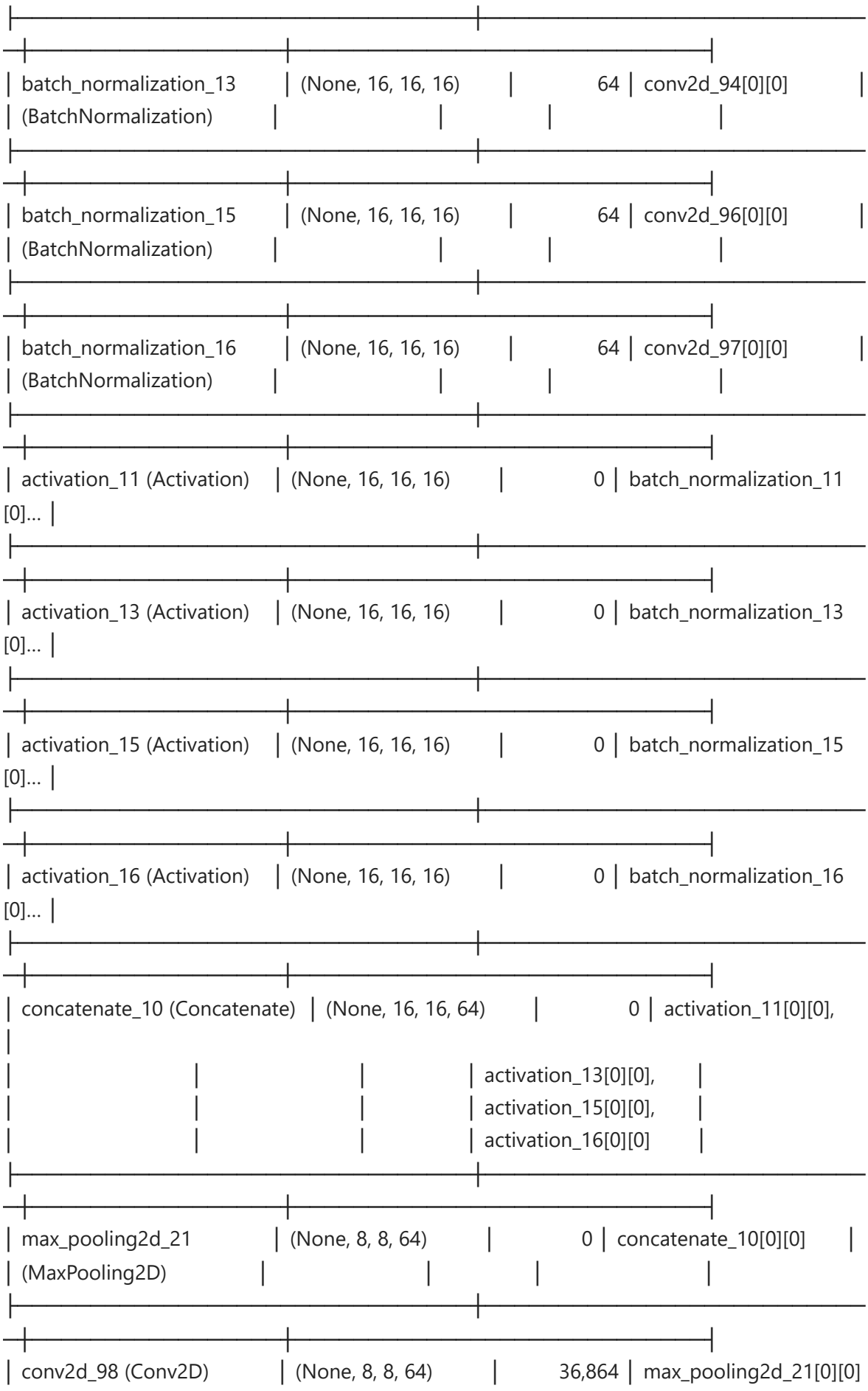
model_c.summary()
print("Trainable parameters:", model_c.count_params())
```

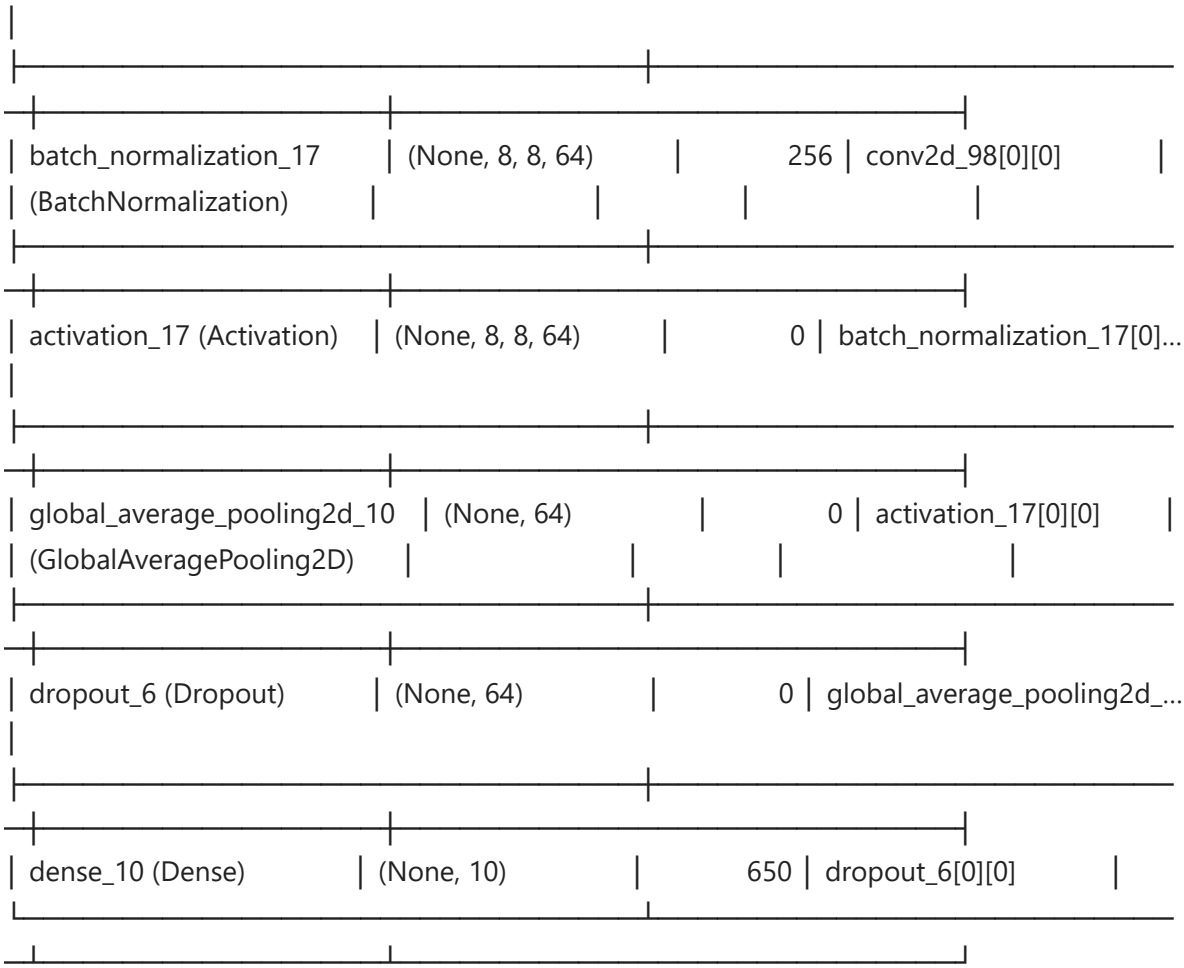
Model: "functional_16"



Layer (type)	Output Shape	Param #	Connected to
input_layer_16 (InputLayer)	(None, 32, 32, 3)	0	-
data_augmentation (Sequential)	(None, 32, 32, 3)	0	input_layer_16[0][0]
conv2d_90 (Conv2D)	(None, 32, 32, 24)	648	data_augmentation[1][0]
batch_normalization_9 (BatchNormalization)	(None, 32, 32, 24)	96	conv2d_90[0][0]
activation_9 (Activation)	(None, 32, 32, 24)	0	batch_normalization_9[0][...
max_pooling2d_20 (MaxPooling2D)	(None, 16, 16, 24)	0	activation_9[0][0]
conv2d_91 (Conv2D)	(None, 16, 16, 32)	6,912	max_pooling2d_20[0][0]
batch_normalization_10 (BatchNormalization)	(None, 16, 16, 32)	128	conv2d_91[0][0]
activation_10 (Activation)	(None, 16, 16, 32)	0	batch_normalization_10[0]...
conv2d_93 (Conv2D)	(None, 16, 16, 16)	512	activation_10[0][0]


conv2d_95 (Conv2D) (None, 16, 16, 16) 512 activation_10[0][0]
batch_normalization_12 (BatchNormalization) (None, 16, 16, 16) 64 conv2d_93[0][0]
batch_normalization_14 (BatchNormalization) (None, 16, 16, 16) 64 conv2d_95[0][0]
activation_12 (Activation) (None, 16, 16, 16) 0 batch_normalization_12[0]...
activation_14 (Activation) (None, 16, 16, 16) 0 batch_normalization_14[0]...
average_pooling2d_10 (AveragePooling2D) (None, 16, 16, 32) 0 activation_10[0][0]
conv2d_92 (Conv2D) (None, 16, 16, 16) 512 activation_10[0][0]
conv2d_94 (Conv2D) (None, 16, 16, 16) 2,304 activation_12[0][0]
conv2d_96 (Conv2D) (None, 16, 16, 16) 6,400 activation_14[0][0]
conv2d_97 (Conv2D) (None, 16, 16, 16) 512 average_pooling2d_10[0][0]
batch_normalization_11 (BatchNormalization) (None, 16, 16, 16) 64 conv2d_92[0][0]


batch_normalization_13 (BatchNormalization) (None, 16, 16, 16) 64 conv2d_94[0][0]
batch_normalization_15 (BatchNormalization) (None, 16, 16, 16) 64 conv2d_96[0][0]
batch_normalization_16 (BatchNormalization) (None, 16, 16, 16) 64 conv2d_97[0][0]
activation_11 (Activation) (None, 16, 16, 16) 0 batch_normalization_11[0]...
activation_13 (Activation) (None, 16, 16, 16) 0 batch_normalization_13[0]...
activation_15 (Activation) (None, 16, 16, 16) 0 batch_normalization_15[0]...
activation_16 (Activation) (None, 16, 16, 16) 0 batch_normalization_16[0]...
concatenate_10 (Concatenate) (None, 16, 16, 64) 0 activation_11[0][0], activation_13[0][0], activation_15[0][0], activation_16[0][0]
max_pooling2d_21 (MaxPooling2D) (None, 8, 8, 64) 0 concatenate_10[0][0]
conv2d_98 (Conv2D) (None, 8, 8, 64) 36,864 max_pooling2d_21[0][0]



Total params: 56,690 (221.45 KB)

Trainable params: 56,258 (219.76 KB)

Non-trainable params: 432 (1.69 KB)

```
Trainable parameters: 56690
```

In [119…
```
y_train_onehot = keras.utils.to_categorical(y_train, 10)
y_val_onehot = keras.utils.to_categorical(y_val, 10)
```

In [120…
```
model_c.compile(
    optimizer=keras.optimizers.AdamW(
        learning_rate=0.001,
        weight_decay=1e-4
    ),
    loss=keras.losses.CategoricalCrossentropy(
        label_smoothing=0.05
    ),
    metrics=["accuracy"]
)
```

In [121…
```
callbacks_c = [

    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=8,
```

```
        restore_best_weights=True,
        verbose=1
    )
]
```

In [122…

```
history_c = model_c.fit(
    x_train,
    y_train_onehot,
    validation_data=(x_val, y_val_onehot),
    epochs=60,
    batch_size=32,
    callbacks=callbacks_c,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 10s 399ms/step - accuracy: 0.0925 - loss: 2.4765 - val_ac
curacy: 0.1000 - val_loss: 2.7737
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.2225 - loss: 2.1678 - val_accu
racy: 0.1000 - val_loss: 2.8705
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.2200 - loss: 2.1092 - val_accu
racy: 0.1400 - val_loss: 2.5175
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.2475 - loss: 2.0423 - val_accu
racy: 0.1600 - val_loss: 2.5528
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.2575 - loss: 2.0161 - val_accu
racy: 0.1700 - val_loss: 2.2798
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step - accuracy: 0.2750 - loss: 1.9816 - val_accu
racy: 0.1900 - val_loss: 2.1972
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step - accuracy: 0.3075 - loss: 1.9280 - val_accu
racy: 0.1900 - val_loss: 2.0928
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step - accuracy: 0.3250 - loss: 1.8866 - val_accu
racy: 0.2400 - val_loss: 2.0330
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - accuracy: 0.2850 - loss: 1.9413 - val_accu
racy: 0.2000 - val_loss: 2.0463
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 78ms/step - accuracy: 0.3175 - loss: 1.8963 - val_accu
racy: 0.1800 - val_loss: 2.1029
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 65ms/step - accuracy: 0.3475 - loss: 1.7920 - val_accu
racy: 0.2800 - val_loss: 2.0279
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 94ms/step - accuracy: 0.3525 - loss: 1.8477 - val_accu
racy: 0.2700 - val_loss: 1.9637
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - accuracy: 0.3375 - loss: 1.8617 - val_accu
racy: 0.2400 - val_loss: 1.9805
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 103ms/step - accuracy: 0.3450 - loss: 1.8015 - val_acc
uracy: 0.3000 - val_loss: 1.8985
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 72ms/step - accuracy: 0.4025 - loss: 1.7383 - val_accu
racy: 0.2800 - val_loss: 1.9414
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 76ms/step - accuracy: 0.4175 - loss: 1.7117 - val_accu
racy: 0.2500 - val_loss: 2.1744
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 76ms/step - accuracy: 0.3925 - loss: 1.7282 - val_accu
racy: 0.3000 - val_loss: 2.0156
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 65ms/step - accuracy: 0.3900 - loss: 1.7287 - val_accu
racy: 0.3000 - val_loss: 1.9677
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - accuracy: 0.4075 - loss: 1.6946 - val_accu
```

```
racy: 0.2800 - val_loss: 1.9219
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 61ms/step - accuracy: 0.4000 - loss: 1.7379 - val_accu
racy: 0.2900 - val_loss: 1.9447
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 57ms/step - accuracy: 0.4300 - loss: 1.6805 - val_accu
racy: 0.2800 - val_loss: 1.9426
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 58ms/step - accuracy: 0.4625 - loss: 1.6488 - val_accu
racy: 0.2800 - val_loss: 1.9940
Epoch 22: early stopping
Restoring model weights from the end of the best epoch: 14.
```

In [123…

```
train_loss_c, train_acc_c = model_c.evaluate(
    x_train, y_train_onehot, verbose=0
)

val_loss_c, val_acc_c = model_c.evaluate(
    x_val, y_val_onehot, verbose=0
)

clean_loss_c, clean_acc_c = model_c.evaluate(
    x_clean, keras.utils.to_categorical(y_clean, 10),
    verbose=1
)

print("Training Accuracy:", train_acc_c * 100, "%")
print("Validation Accuracy:", val_acc_c * 100, "%")
print("Clean Test Accuracy:", clean_acc_c * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.3240 - loss: 1.9084
Training Accuracy: 44.24999952316284 %
Validation Accuracy: 30.000001192092896 %
Clean Test Accuracy: 32.40000009536743 %
```

In [124…

```
y_shifted_onehot = keras.utils.to_categorical(y_shifted, 10)

shifted_loss_c, shifted_acc_c = model_c.evaluate(
    x_shifted, y_shifted_onehot, verbose=1
)

print("Synthetic Shifted Accuracy:", shifted_acc_c * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 15ms/step - accuracy: 0.2060 - loss: 2.1401
Synthetic Shifted Accuracy: 20.600000023841858 %
```

<h3 id="Regularization-and-Training-Design">Regularization and Training Design</h3>
Batch Normalization is used after convolutional layers to make training more stable. Dropout is added after Global Average Pooling to help prevent overfitting. AdamW uses weight decay to prevent the model's weights from becoming too large. Label smoothing helps the model make less overconfident predictions. EarlyStopping stops training when the validation loss does not improve and restores the best model weights.

PART D

In [125…

```
def build_model(augmentation=None):
    inputs = keras.Input(shape=(32, 32, 3))

    x = inputs

    if augmentation is not None:
        x = augmentation(x)

    x = layers.Conv2D(
        24, 3, padding="same", activation="relu"
    )(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(
        32, 3, padding="same", activation="relu"
    )(x)

    x = inception_module(x, filters=16)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(
        64, 3, padding="same", activation="relu"
    )(x)

    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.3)(x)

    outputs = layers.Dense(
        10, activation="softmax"
    )(x)

    model = keras.Model(inputs, outputs)

    model.compile(
        optimizer=keras.optimizers.Adam(0.001),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    return model
```

In [126…

```
geometric_aug = keras.Sequential([
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(25 / 360)
])

tf.keras.utils.set_random_seed(42)

model_geo = build_model(geometric_aug)

history_geo = model_geo.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=60,
```

```
    batch_size=32,
    verbose=1
)

val_loss, val_acc = model_geo.evaluate(
    x_val, y_val, verbose=0
)

print("Geometric validation accuracy:", val_acc)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 7s 368ms/step - accuracy: 0.1300 - loss: 5.5535 - val_acc
uracy: 0.1800 - val_loss: 2.2963
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.1250 - loss: 2.2766 - val_accu
racy: 0.1800 - val_loss: 2.2439
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.1575 - loss: 2.2418 - val_accu
racy: 0.1900 - val_loss: 2.1782
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.1725 - loss: 2.2083 - val_accu
racy: 0.2500 - val_loss: 2.1220
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.1550 - loss: 2.1767 - val_accu
racy: 0.2500 - val_loss: 2.1015
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.1800 - loss: 2.1615 - val_accu
racy: 0.1400 - val_loss: 2.0922
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.2100 - loss: 2.1406 - val_accu
racy: 0.2800 - val_loss: 2.0482
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.2300 - loss: 2.0742 - val_accu
racy: 0.2100 - val_loss: 2.0420
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2300 - loss: 2.0369 - val_accu
racy: 0.2100 - val_loss: 1.9580
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2175 - loss: 2.0474 - val_accu
racy: 0.3500 - val_loss: 1.9643
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2400 - loss: 1.9926 - val_accu
racy: 0.2100 - val_loss: 1.9856
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2725 - loss: 2.0179 - val_accu
racy: 0.2700 - val_loss: 1.9510
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.2325 - loss: 1.9784 - val_accu
racy: 0.2700 - val_loss: 1.8720
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.3150 - loss: 1.9266 - val_accu
racy: 0.3600 - val_loss: 1.8332
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.2575 - loss: 1.9401 - val_accu
racy: 0.1900 - val_loss: 1.9663
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.2700 - loss: 1.9062 - val_accu
racy: 0.3000 - val_loss: 1.8158
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.2825 - loss: 1.9092 - val_accu
racy: 0.3400 - val_loss: 1.8138
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2750 - loss: 1.8637 - val_accu
racy: 0.2800 - val_loss: 1.8427
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.3125 - loss: 1.8484 - val_accu
```

```
racy: 0.3100 - val_loss: 1.7850
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.2975 - loss: 1.8522 - val_accu
racy: 0.3100 - val_loss: 1.7788
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step - accuracy: 0.3475 - loss: 1.8063 - val_accu
racy: 0.3100 - val_loss: 1.7085
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3225 - loss: 1.8113 - val_accu
racy: 0.2800 - val_loss: 1.8750
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3025 - loss: 1.8342 - val_accu
racy: 0.3800 - val_loss: 1.7181
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3550 - loss: 1.7544 - val_accu
racy: 0.4100 - val_loss: 1.6925
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.3775 - loss: 1.6872 - val_accu
racy: 0.3800 - val_loss: 1.6564
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3675 - loss: 1.7559 - val_accu
racy: 0.3700 - val_loss: 1.8131
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.3900 - loss: 1.7045 - val_accu
racy: 0.3200 - val_loss: 1.6306
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.3275 - loss: 1.7234 - val_accu
racy: 0.4100 - val_loss: 1.6500
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.3475 - loss: 1.6671 - val_accu
racy: 0.3400 - val_loss: 1.6744
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 58ms/step - accuracy: 0.3775 - loss: 1.6598 - val_accu
racy: 0.3500 - val_loss: 1.7411
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.3650 - loss: 1.7268 - val_accu
racy: 0.3100 - val_loss: 1.6419
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 56ms/step - accuracy: 0.3375 - loss: 1.7092 - val_accu
racy: 0.3300 - val_loss: 1.7729
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 66ms/step - accuracy: 0.4000 - loss: 1.6422 - val_accu
racy: 0.3700 - val_loss: 1.7114
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.3775 - loss: 1.6348 - val_accu
racy: 0.4200 - val_loss: 1.6951
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.3950 - loss: 1.6448 - val_accu
racy: 0.4000 - val_loss: 1.6848
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.4075 - loss: 1.6323 - val_accu
racy: 0.3600 - val_loss: 1.6689
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 56ms/step - accuracy: 0.4075 - loss: 1.6675 - val_accu
racy: 0.3700 - val_loss: 1.8529
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.3800 - loss: 1.6671 - val_accu
racy: 0.3400 - val_loss: 1.7271
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.3825 - loss: 1.6162 - val_accu
racy: 0.3300 - val_loss: 1.6525
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3900 - loss: 1.5400 - val_accu
racy: 0.3300 - val_loss: 1.7905
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 68ms/step - accuracy: 0.4275 - loss: 1.5731 - val_accu
racy: 0.3600 - val_loss: 1.6480
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.4200 - loss: 1.5599 - val_accu
racy: 0.4000 - val_loss: 1.6031
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.4250 - loss: 1.5719 - val_accu
racy: 0.3600 - val_loss: 1.6157
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.4200 - loss: 1.4932 - val_accu
racy: 0.3300 - val_loss: 1.6751
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.4450 - loss: 1.5054 - val_accu
racy: 0.3700 - val_loss: 1.6004
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.4600 - loss: 1.4728 - val_accu
racy: 0.3800 - val_loss: 1.6830
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.4200 - loss: 1.5233 - val_accu
racy: 0.3600 - val_loss: 1.6974
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.4200 - loss: 1.5744 - val_accu
racy: 0.4500 - val_loss: 1.6643
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step - accuracy: 0.4350 - loss: 1.4865 - val_accu
racy: 0.3600 - val_loss: 1.6950
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.4450 - loss: 1.5021 - val_accu
racy: 0.3700 - val_loss: 1.7319
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.4575 - loss: 1.4755 - val_accu
racy: 0.3500 - val_loss: 1.6558
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.4125 - loss: 1.5192 - val_accu
racy: 0.4000 - val_loss: 1.8046
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.4275 - loss: 1.5254 - val_accu
racy: 0.3800 - val_loss: 1.6591
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.5050 - loss: 1.3885 - val_accu
racy: 0.4100 - val_loss: 1.6298
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.4400 - loss: 1.4364 - val_accu
racy: 0.3300 - val_loss: 1.7673
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 58ms/step - accuracy: 0.4800 - loss: 1.4258 - val_accu
racy: 0.3700 - val_loss: 1.7483
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - accuracy: 0.4975 - loss: 1.3967 - val_accu
racy: 0.3500 - val_loss: 1.6599
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.4625 - loss: 1.3840 - val_accu
racy: 0.4000 - val_loss: 1.6958
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4800 - loss: 1.4257 - val_accu
racy: 0.4100 - val_loss: 1.6611
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.4800 - loss: 1.4270 - val_accu
racy: 0.3800 - val_loss: 1.8100
Geometric validation accuracy: 0.3799999952316284
```

In [127…

```
results = [
    "No augmentation",
    "Geometric only",
    "Photometric only",
    "Geometric + photometric",
    "+ Cutout / Random Erasing",
    "+ Mixup / CutMix + label smoothing",
    "Final model"
]

df_results = pd.DataFrame({
    "Configuration": results,
    "Val Acc": [None] * 7,
    "Clean Test": [None] * 7,
    "Shifted Test (Mean ± Std)": [None] * 7
})

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

0	No augmentation	None	None	None
1	Geometric only	None	None	None
2	Photometric only	None	None	None
3	Geometric + photometric	None	None	None
4	+ Cutout / Random Erasing	None	None	None
5	+ Mixup / CutMix + label smoothing	None	None	None
6	Final model	None	None	None

In [128…

```
_, val_acc = model_geo.evaluate(
    x_val, y_val, verbose=0
)
```

```
_, clean_acc = model_geo.evaluate(
    x_clean, y_clean, verbose=0
)

print("Validation Accuracy:", round(val_acc * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc * 100, 2), "%")
```

```
Validation Accuracy: 38.0 %
Clean Test Accuracy: 34.38 %
```

In [129…

```
_, shifted_acc = model_geo.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Shifted Test Accuracy:",
      round(shifted_acc * 100, 2), "%")
```

```
Shifted Test Accuracy: 23.74 %
```

In [130…

```
df_results.loc[1, "Val Acc"] = round(
    val_acc * 100, 2
)

df_results.loc[1, "Clean Test"] = round(
    clean_acc * 100, 2
)
df_results.loc[1, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc_geo * 100:.2f}% (seed 2026)"
)
display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	None	None	None
Geometric only	38.0	34.38	23.74% (seed 2026)
Photometric only	None	None	None
Geometric + photometric	None	None	None
+ Cutout / Random Erasing	None	None	None
+ Mixup / CutMix + label smoothing	None	None	None
Final model	None	None	None

In [131…

```
_, shifted_acc_geo = model_geo.evaluate(
    x_shifted,
    y_shifted,
    verbose=0
)
```

```
print("Shifted accuracy:", shifted_acc_geo)
print("Shifted accuracy (%):", shifted_acc_geo * 100)
```

```
Shifted accuracy: 0.23739999532699585
Shifted accuracy (%): 23.739999532699585
```

In [132…

```
_, shifted_acc_geo = model_geo.evaluate(
    x_shifted,
    y_shifted,
    verbose=0
)

print("Shifted accuracy:", shifted_acc_geo)
print("Shifted accuracy (%):", shifted_acc_geo * 100)
```

```
Shifted accuracy: 0.23739999532699585
Shifted accuracy (%): 23.739999532699585
```

In [133…

```
tf.keras.utils.set_random_seed(SEED)

photometric_aug = keras.Sequential([
    layers.RandomBrightness(
        factor=0.2,
        value_range=(0, 255)
    ),
    layers.RandomContrast(factor=0.2)
])
```

In [134…

```
model_photo = build_model(photometric_aug)

history_photo = model_photo.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 4s 171ms/step - accuracy: 0.1175 - loss: 10.5436 - val_ac
curacy: 0.1600 - val_loss: 2.3231
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.1025 - loss: 2.3348 - val_accu
racy: 0.1200 - val_loss: 2.2639
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.1325 - loss: 2.2854 - val_accu
racy: 0.1600 - val_loss: 2.2462
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.1900 - loss: 2.2308 - val_accu
racy: 0.1900 - val_loss: 2.1902
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.1525 - loss: 2.2353 - val_accu
racy: 0.2200 - val_loss: 2.1613
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.1775 - loss: 2.1835 - val_accu
racy: 0.1800 - val_loss: 2.1124
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2100 - loss: 2.1525 - val_accu
racy: 0.2600 - val_loss: 2.0581
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2125 - loss: 2.1583 - val_accu
racy: 0.2300 - val_loss: 2.0767
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2225 - loss: 2.0729 - val_accu
racy: 0.2100 - val_loss: 2.0131
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2250 - loss: 2.0398 - val_accu
racy: 0.2500 - val_loss: 1.9213
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2300 - loss: 2.0687 - val_accu
racy: 0.2600 - val_loss: 2.0124
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2675 - loss: 2.0179 - val_accu
racy: 0.2700 - val_loss: 1.9197
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.2775 - loss: 1.9751 - val_accu
racy: 0.2500 - val_loss: 1.9308
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.2625 - loss: 1.9671 - val_accu
racy: 0.3000 - val_loss: 1.9212
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2325 - loss: 1.9309 - val_accu
racy: 0.3500 - val_loss: 1.8500
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.2900 - loss: 1.9614 - val_accu
racy: 0.2900 - val_loss: 1.9229
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2875 - loss: 1.8722 - val_accu
racy: 0.3200 - val_loss: 1.8678
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.2650 - loss: 1.8644 - val_accu
racy: 0.2400 - val_loss: 1.8830
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3250 - loss: 1.7950 - val_accu
```

```
racy: 0.3200 - val_loss: 1.8216
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3150 - loss: 1.8116 - val_accu
racy: 0.3600 - val_loss: 1.8067
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3750 - loss: 1.7613 - val_accu
racy: 0.2900 - val_loss: 1.8614
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3500 - loss: 1.7193 - val_accu
racy: 0.2700 - val_loss: 1.7853
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3600 - loss: 1.7080 - val_accu
racy: 0.2600 - val_loss: 1.8470
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3400 - loss: 1.7484 - val_accu
racy: 0.3400 - val_loss: 1.7839
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3400 - loss: 1.7017 - val_accu
racy: 0.3200 - val_loss: 1.8277
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4000 - loss: 1.6361 - val_accu
racy: 0.3000 - val_loss: 1.8038
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.4100 - loss: 1.5951 - val_accu
racy: 0.3600 - val_loss: 1.7340
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3925 - loss: 1.6243 - val_accu
racy: 0.3000 - val_loss: 1.7765
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4075 - loss: 1.5842 - val_accu
racy: 0.3500 - val_loss: 1.7697
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4350 - loss: 1.5375 - val_accu
racy: 0.3100 - val_loss: 1.6614
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4225 - loss: 1.5626 - val_accu
racy: 0.2800 - val_loss: 1.7712
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4475 - loss: 1.4835 - val_accu
racy: 0.3200 - val_loss: 1.7529
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4775 - loss: 1.5332 - val_accu
racy: 0.3300 - val_loss: 1.6762
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4625 - loss: 1.4703 - val_accu
racy: 0.2800 - val_loss: 1.6586
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4625 - loss: 1.4519 - val_accu
racy: 0.3300 - val_loss: 1.6957
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5050 - loss: 1.3515 - val_accu
racy: 0.2400 - val_loss: 1.7712
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 56ms/step - accuracy: 0.5000 - loss: 1.3554 - val_accu
racy: 0.3600 - val_loss: 1.7441
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5025 - loss: 1.3653 - val_accu
racy: 0.2100 - val_loss: 1.8301
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5125 - loss: 1.3238 - val_accu
racy: 0.3500 - val_loss: 1.6869
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5375 - loss: 1.2698 - val_accu
racy: 0.3300 - val_loss: 1.7191
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5275 - loss: 1.2938 - val_accu
racy: 0.2700 - val_loss: 1.7075
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5675 - loss: 1.2677 - val_accu
racy: 0.3000 - val_loss: 1.7961
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5300 - loss: 1.2534 - val_accu
racy: 0.3100 - val_loss: 1.6947
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5675 - loss: 1.1711 - val_accu
racy: 0.3500 - val_loss: 1.6891
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.6000 - loss: 1.1312 - val_accu
racy: 0.3100 - val_loss: 1.7202
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6100 - loss: 1.1708 - val_accu
racy: 0.2700 - val_loss: 1.7567
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5850 - loss: 1.1332 - val_accu
racy: 0.3400 - val_loss: 1.7348
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.6250 - loss: 1.0754 - val_accu
racy: 0.3100 - val_loss: 1.7385
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6475 - loss: 1.0372 - val_accu
racy: 0.3100 - val_loss: 1.8274
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6275 - loss: 1.0467 - val_accu
racy: 0.4000 - val_loss: 1.7074
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6475 - loss: 0.9394 - val_accu
racy: 0.3400 - val_loss: 1.7713
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6300 - loss: 1.0252 - val_accu
racy: 0.3400 - val_loss: 1.7194
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6375 - loss: 0.9951 - val_accu
racy: 0.3000 - val_loss: 1.6858
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.7225 - loss: 0.8251 - val_accu
racy: 0.3100 - val_loss: 1.8765
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.6350 - loss: 0.9407 - val_accu
racy: 0.4200 - val_loss: 1.7052
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6600 - loss: 0.9540 - val_accu
racy: 0.3300 - val_loss: 1.6519
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.6725 - loss: 0.8879 - val_accu
racy: 0.3300 - val_loss: 1.7132
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6875 - loss: 0.8640 - val_accu
racy: 0.3400 - val_loss: 1.7896
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6750 - loss: 0.8752 - val_accu
racy: 0.3300 - val_loss: 1.6912
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.6800 - loss: 0.8950 - val_accu
racy: 0.3500 - val_loss: 1.6945
```

In [135…

```
_, val_acc = model_photo.evaluate(
    x_val, y_val, verbose=0
)

_, clean_acc = model_photo.evaluate(
    x_clean, y_clean, verbose=0
)

_, shifted_acc = model_photo.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Validation Accuracy:", round(val_acc * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc * 100, 2), "%")
print("Shifted Test Accuracy:", round(shifted_acc * 100, 2), "%")
```

```
Validation Accuracy: 35.0 %
Clean Test Accuracy: 36.89 %
Shifted Test Accuracy: 18.83 %
```

In [136…

```
df_results.loc[2, "Val Acc"] = round(val_acc * 100, 2)

df_results.loc[2, "Clean Test"] = round(clean_acc * 100, 2)

df_results.loc[2, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc * 100:.2f}% (seed 2026)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	None	None	None
Geometric only	38.0	34.38	23.74% (seed 2026)
Photometric only	35.0	36.89	18.83% (seed 2026)
Geometric + photometric	None	None	None
+ Cutout / Random Erasing	None	None	None
+ Mixup / CutMix + label smoothing	None	None	None
Final model	None	None	None

In [137…

```
tf.keras.utils.set_random_seed(SEED)

combined_aug = keras.Sequential([
    layers.RandomTranslation(
        0.1, 0.1,
        fill_mode="nearest"
    ),

    layers.RandomFlip("horizontal"),

    layers.RandomRotation(
        25 / 360,
        fill_mode="nearest"
    ),

    layers.RandomBrightness(
        factor=0.2,
        value_range=(0, 255)
    ),

    layers.RandomContrast(
        factor=0.2
    )
])
```

In [138…

```
model_combined = build_model(combined_aug)

history_combined = model_combined.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 202ms/step - accuracy: 0.0875 - loss: 7.6093 - val_acc
uracy: 0.1800 - val_loss: 2.3228
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1350 - loss: 2.3194 - val_accu
racy: 0.1100 - val_loss: 2.2506
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1600 - loss: 2.2635 - val_accu
racy: 0.1300 - val_loss: 2.2389
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1400 - loss: 2.2693 - val_accu
racy: 0.1700 - val_loss: 2.2158
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1700 - loss: 2.2247 - val_accu
racy: 0.1800 - val_loss: 2.1748
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1925 - loss: 2.1707 - val_accu
racy: 0.1900 - val_loss: 2.1201
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1700 - loss: 2.2143 - val_accu
racy: 0.2400 - val_loss: 2.1014
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2075 - loss: 2.1394 - val_accu
racy: 0.2600 - val_loss: 2.0416
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.2150 - loss: 2.1369 - val_accu
racy: 0.2100 - val_loss: 2.0282
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - accuracy: 0.1825 - loss: 2.1085 - val_accu
racy: 0.2300 - val_loss: 1.9981
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.2025 - loss: 2.0927 - val_accu
racy: 0.2500 - val_loss: 1.9540
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.2325 - loss: 2.0815 - val_accu
racy: 0.2300 - val_loss: 1.9441
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.2325 - loss: 2.0712 - val_accu
racy: 0.2700 - val_loss: 1.9266
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - accuracy: 0.2575 - loss: 2.0220 - val_accu
racy: 0.3000 - val_loss: 1.8820
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - accuracy: 0.2350 - loss: 2.0152 - val_accu
racy: 0.2900 - val_loss: 1.8815
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.2675 - loss: 1.9817 - val_accu
racy: 0.2800 - val_loss: 1.8641
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.2825 - loss: 1.9559 - val_accu
racy: 0.2900 - val_loss: 1.8525
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2500 - loss: 1.9962 - val_accu
racy: 0.3100 - val_loss: 1.7968
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2725 - loss: 1.9228 - val_accu
```

```
racy: 0.3400 - val_loss: 1.8113
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2550 - loss: 1.9353 - val_accu
racy: 0.3800 - val_loss: 1.8140
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.3150 - loss: 1.8977 - val_accu
racy: 0.3600 - val_loss: 1.7688
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.2725 - loss: 1.9094 - val_accu
racy: 0.3100 - val_loss: 1.8109
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.3050 - loss: 1.8690 - val_accu
racy: 0.3500 - val_loss: 1.7893
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2850 - loss: 1.9287 - val_accu
racy: 0.4100 - val_loss: 1.7694
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.3275 - loss: 1.8637 - val_accu
racy: 0.3500 - val_loss: 1.7997
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3150 - loss: 1.8594 - val_accu
racy: 0.3400 - val_loss: 1.7602
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.3075 - loss: 1.8684 - val_accu
racy: 0.2900 - val_loss: 1.7605
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.3000 - loss: 1.8428 - val_accu
racy: 0.3700 - val_loss: 1.7357
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.3175 - loss: 1.8103 - val_accu
racy: 0.3700 - val_loss: 1.7125
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2850 - loss: 1.8561 - val_accu
racy: 0.3600 - val_loss: 1.7148
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.3200 - loss: 1.8006 - val_accu
racy: 0.4300 - val_loss: 1.6849
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3225 - loss: 1.7966 - val_accu
racy: 0.3700 - val_loss: 1.6860
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.3250 - loss: 1.7436 - val_accu
racy: 0.3900 - val_loss: 1.6926
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.3300 - loss: 1.7289 - val_accu
racy: 0.4000 - val_loss: 1.6745
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.3450 - loss: 1.7583 - val_accu
racy: 0.3200 - val_loss: 1.7306
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.3375 - loss: 1.7700 - val_accu
racy: 0.3300 - val_loss: 1.6821
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.3225 - loss: 1.7461 - val_accu
racy: 0.4200 - val_loss: 1.6277
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.3900 - loss: 1.7214 - val_accu
racy: 0.3400 - val_loss: 1.6722
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.3450 - loss: 1.6914 - val_accu
racy: 0.4000 - val_loss: 1.6996
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3250 - loss: 1.7961 - val_accu
racy: 0.3600 - val_loss: 1.7265
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 64ms/step - accuracy: 0.3850 - loss: 1.7342 - val_accu
racy: 0.3200 - val_loss: 1.6832
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.3500 - loss: 1.7297 - val_accu
racy: 0.3900 - val_loss: 1.6924
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.3825 - loss: 1.7091 - val_accu
racy: 0.3800 - val_loss: 1.6172
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.3825 - loss: 1.6540 - val_accu
racy: 0.3800 - val_loss: 1.6395
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.3725 - loss: 1.6990 - val_accu
racy: 0.3100 - val_loss: 1.7373
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.4125 - loss: 1.6169 - val_accu
racy: 0.3600 - val_loss: 1.6517
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.3750 - loss: 1.6765 - val_accu
racy: 0.4500 - val_loss: 1.5755
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.4350 - loss: 1.5974 - val_accu
racy: 0.3500 - val_loss: 1.6380
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.4150 - loss: 1.5724 - val_accu
racy: 0.3700 - val_loss: 1.6881
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.4050 - loss: 1.6680 - val_accu
racy: 0.4000 - val_loss: 1.6563
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.4175 - loss: 1.6202 - val_accu
racy: 0.3600 - val_loss: 1.6609
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.3700 - loss: 1.6512 - val_accu
racy: 0.3600 - val_loss: 1.6944
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.4575 - loss: 1.6061 - val_accu
racy: 0.3800 - val_loss: 1.6456
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.4275 - loss: 1.5792 - val_accu
racy: 0.3300 - val_loss: 1.6634
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.4475 - loss: 1.5746 - val_accu
racy: 0.3400 - val_loss: 1.6776
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.4525 - loss: 1.5337 - val_accu
racy: 0.3200 - val_loss: 1.7252
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.4150 - loss: 1.6055 - val_accu
racy: 0.4000 - val_loss: 1.6348
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.4300 - loss: 1.5426 - val_accu
racy: 0.3800 - val_loss: 1.6123
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.4200 - loss: 1.5124 - val_accu
racy: 0.3500 - val_loss: 1.6632
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - accuracy: 0.4375 - loss: 1.4892 - val_accu
racy: 0.3800 - val_loss: 1.6043
```

In [139…

```python
_, val_acc = model_combined.evaluate(
    x_val, y_val, verbose=0
)

_, clean_acc = model_combined.evaluate(
    x_clean, y_clean, verbose=0
)

_, shifted_acc = model_combined.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Validation Accuracy:", round(val_acc * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc * 100, 2), "%")
print("Shifted Test Accuracy:", round(shifted_acc * 100, 2), "%")
```

```
Validation Accuracy: 38.0 %
Clean Test Accuracy: 37.16 %
Shifted Test Accuracy: 22.5 %
```

In [140…

```python
df_results.loc[3, "Val Acc"] = round(
    val_acc * 100, 2
)

df_results.loc[3, "Clean Test"] = round(
    clean_acc * 100, 2
)

df_results.loc[3, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc * 100:.2f}% (seed 2026)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	None	None	None
Geometric only	38.0	34.38	23.74% (seed 2026)
Photometric only	35.0	36.89	18.83% (seed 2026)
Geometric + photometric	38.0	37.16	22.50% (seed 2026)
+ Cutout / Random Erasing	None	None	None
+ Mixup / CutMix + label smoothing	None	None	None
Final model	None	None	None

In [141…

```
def apply_cutout(images):
    batch_size = tf.shape(images)[0]

    top = tf.random.uniform(
        [batch_size], 0, 17, dtype=tf.int32
    )
    left = tf.random.uniform(
        [batch_size], 0, 17, dtype=tf.int32
    )

    rows = tf.range(32)[None, :, None]
    cols = tf.range(32)[None, None, :]

    mask = (
        (rows < top[:, None, None]) |
        (rows >= (top + 16)[:, None, None]) |
        (cols < left[:, None, None]) |
        (cols >= (left + 16)[:, None, None])
    )

    mask = tf.cast(mask[..., None], images.dtype)

    return images * mask
```

In [142…

```
tf.keras.utils.set_random_seed(SEED)

cutout_aug = keras.Sequential([
    combined_aug,
    layers.Lambda(apply_cutout)
])
```

In [143…

```
model_cutout = build_model(cutout_aug)

history_cutout = model_cutout.fit(
    x_train,
```

```
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 6s 241ms/step - accuracy: 0.0950 - loss: 8.7042 - val_acc
uracy: 0.0900 - val_loss: 2.3677
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.1050 - loss: 2.4037 - val_accu
racy: 0.1200 - val_loss: 2.2930
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.1300 - loss: 2.2930 - val_accu
racy: 0.1100 - val_loss: 2.2748
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.1200 - loss: 2.2903 - val_accu
racy: 0.1700 - val_loss: 2.2666
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.1450 - loss: 2.2828 - val_accu
racy: 0.1700 - val_loss: 2.2547
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - accuracy: 0.1750 - loss: 2.2817 - val_accu
racy: 0.1900 - val_loss: 2.2310
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.1675 - loss: 2.2677 - val_accu
racy: 0.1400 - val_loss: 2.2289
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.1650 - loss: 2.2532 - val_accu
racy: 0.2000 - val_loss: 2.2060
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.1875 - loss: 2.2261 - val_accu
racy: 0.1800 - val_loss: 2.1989
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.1425 - loss: 2.2470 - val_accu
racy: 0.2600 - val_loss: 2.1837
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.1825 - loss: 2.2310 - val_accu
racy: 0.2200 - val_loss: 2.1865
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.1725 - loss: 2.2443 - val_accu
racy: 0.2100 - val_loss: 2.1942
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.1825 - loss: 2.2339 - val_accu
racy: 0.1800 - val_loss: 2.1591
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.1975 - loss: 2.1976 - val_accu
racy: 0.1900 - val_loss: 2.1142
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.1575 - loss: 2.2087 - val_accu
racy: 0.1900 - val_loss: 2.1405
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.1725 - loss: 2.2048 - val_accu
racy: 0.1900 - val_loss: 2.1216
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.1900 - loss: 2.1705 - val_accu
racy: 0.3100 - val_loss: 2.0966
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.1950 - loss: 2.1303 - val_accu
racy: 0.2500 - val_loss: 2.0430
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.1550 - loss: 2.1797 - val_accu
```

```
racy: 0.2100 - val_loss: 2.0800
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2050 - loss: 2.1649 - val_accu
racy: 0.2700 - val_loss: 2.0901
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.2025 - loss: 2.1247 - val_accu
racy: 0.2400 - val_loss: 2.0426
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2300 - loss: 2.1112 - val_accu
racy: 0.2700 - val_loss: 1.9941
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2125 - loss: 2.1243 - val_accu
racy: 0.2900 - val_loss: 2.0500
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2375 - loss: 2.0777 - val_accu
racy: 0.3100 - val_loss: 2.0048
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2475 - loss: 2.1076 - val_accu
racy: 0.2400 - val_loss: 2.0688
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.1850 - loss: 2.1130 - val_accu
racy: 0.2600 - val_loss: 2.0482
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.1975 - loss: 2.0809 - val_accu
racy: 0.3200 - val_loss: 2.0063
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2075 - loss: 2.0830 - val_accu
racy: 0.2400 - val_loss: 1.9944
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2350 - loss: 2.0747 - val_accu
racy: 0.3200 - val_loss: 1.9828
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2450 - loss: 2.0514 - val_accu
racy: 0.2600 - val_loss: 2.0211
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2200 - loss: 2.0762 - val_accu
racy: 0.2500 - val_loss: 2.0777
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2175 - loss: 2.0751 - val_accu
racy: 0.2500 - val_loss: 2.0011
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2475 - loss: 2.0479 - val_accu
racy: 0.2900 - val_loss: 1.9279
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2600 - loss: 2.0416 - val_accu
racy: 0.2300 - val_loss: 2.0089
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - accuracy: 0.2325 - loss: 2.0107 - val_accu
racy: 0.3300 - val_loss: 1.9209
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2425 - loss: 2.0201 - val_accu
racy: 0.3300 - val_loss: 1.8538
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2450 - loss: 2.0068 - val_accu
racy: 0.3100 - val_loss: 1.9234
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2750 - loss: 1.9613 - val_accu
racy: 0.3600 - val_loss: 1.8392
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.2575 - loss: 1.9889 - val_accu
racy: 0.2700 - val_loss: 1.8811
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.2625 - loss: 1.9473 - val_accu
racy: 0.3400 - val_loss: 1.8628
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.2425 - loss: 1.9579 - val_accu
racy: 0.3200 - val_loss: 1.8725
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2300 - loss: 1.9726 - val_accu
racy: 0.3400 - val_loss: 1.8691
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.2650 - loss: 1.9454 - val_accu
racy: 0.3800 - val_loss: 1.8400
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.2550 - loss: 1.9228 - val_accu
racy: 0.3100 - val_loss: 1.8970
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2875 - loss: 1.9334 - val_accu
racy: 0.3100 - val_loss: 1.8265
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.2375 - loss: 1.9384 - val_accu
racy: 0.3500 - val_loss: 1.8092
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.2675 - loss: 1.9311 - val_accu
racy: 0.3500 - val_loss: 1.8064
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2900 - loss: 1.8986 - val_accu
racy: 0.3700 - val_loss: 1.7833
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2500 - loss: 1.9598 - val_accu
racy: 0.3400 - val_loss: 1.8648
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2850 - loss: 1.9307 - val_accu
racy: 0.3100 - val_loss: 1.8420
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - accuracy: 0.3225 - loss: 1.9008 - val_accu
racy: 0.3400 - val_loss: 1.8361
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.2850 - loss: 1.9421 - val_accu
racy: 0.3500 - val_loss: 1.7917
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.3175 - loss: 1.9367 - val_accu
racy: 0.3300 - val_loss: 1.8222
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.2575 - loss: 1.9399 - val_accu
racy: 0.3200 - val_loss: 1.8000
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - accuracy: 0.2875 - loss: 1.8776 - val_accu
racy: 0.4100 - val_loss: 1.7530
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.3075 - loss: 1.8945 - val_accu
racy: 0.3600 - val_loss: 1.7722
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.2775 - loss: 1.9240 - val_accu
racy: 0.4000 - val_loss: 1.7584
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.3000 - loss: 1.8840 - val_accu
racy: 0.3600 - val_loss: 1.7639
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.3050 - loss: 1.9334 - val_accu
racy: 0.3600 - val_loss: 1.7910
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.2850 - loss: 1.8789 - val_accu
racy: 0.3900 - val_loss: 1.7540
```

In [144…

```python
_, val_acc = model_cutout.evaluate(
    x_val, y_val, verbose=0
)

_, clean_acc = model_cutout.evaluate(
    x_clean, y_clean, verbose=0
)

_, shifted_acc = model_cutout.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Validation Accuracy:", round(val_acc * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc * 100, 2), "%")
print("Synthetic Shifted Accuracy:", round(shifted_acc * 100, 2), "%")
```

```
Validation Accuracy: 39.0 %
Clean Test Accuracy: 30.93 %
Synthetic Shifted Accuracy: 28.57 %
```

In [145…

```python
df_results.loc[4, "Val Acc"] = round(val_acc * 100, 2)
df_results.loc[4, "Clean Test"] = round(clean_acc * 100, 2)

df_results.loc[4, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc * 100:.2f}% (seed 2026)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	None	None	None
Geometric only	38.0	34.38	23.74% (seed 2026)
Photometric only	35.0	36.89	18.83% (seed 2026)
Geometric + photometric	38.0	37.16	22.50% (seed 2026)
+ Cutout / Random Erasing	39.0	30.93	28.57% (seed 2026)
+ Mixup / CutMix + label smoothing	None	None	None
Final model	None	None	None

In [146…

```
def mixup_batch(images, labels, alpha=0.2):
    lam = np.random.beta(alpha, alpha)

    indices = tf.random.shuffle(
        tf.range(tf.shape(images)[0])
    )

    mixed_images = (
        lam * images +
        (1 - lam) * tf.gather(images, indices)
    )

    mixed_labels = (
        lam * labels +
        (1 - lam) * tf.gather(labels, indices)
    )

    return mixed_images, mixed_labels
```

In [147…

```
y_train_onehot = keras.utils.to_categorical(
    y_train, 10
)

y_val_onehot = keras.utils.to_categorical(
    y_val, 10
)
```

In [148…

```
tf.keras.utils.set_random_seed(SEED)

model_mixup = build_model(combined_aug)

model_mixup.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=0.001
    ),
```

```
    loss=keras.losses.CategoricalCrossentropy(
        label_smoothing=0.1
    ),
    metrics=["accuracy"]
)
```

In [149…

```
_, val_acc = model_mixup.evaluate(
    x_val, y_val_onehot, verbose=0
)

_, clean_acc = model_mixup.evaluate(
    x_clean,
    keras.utils.to_categorical(y_clean, 10),
    verbose=0
)

_, shifted_acc = model_mixup.evaluate(
    x_shifted,
    keras.utils.to_categorical(y_shifted, 10),
    verbose=0
)

print("Validation Accuracy:", round(val_acc * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc * 100, 2), "%")
print("Synthetic Shifted Accuracy:", round(shifted_acc * 100, 2), "%")
```

```
Validation Accuracy: 7.0 %
Clean Test Accuracy: 9.83 %
Synthetic Shifted Accuracy: 10.86 %
```

In [150…

```
df_results.loc[5, "Val Acc"] = round(val_acc * 100, 2)
df_results.loc[5, "Clean Test"] = round(clean_acc * 100, 2)

df_results.loc[5, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc * 100:.2f}% (seed 2026)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	None	None	None
Geometric only	38.0	34.38	23.74% (seed 2026)
Photometric only	35.0	36.89	18.83% (seed 2026)
Geometric + photometric	38.0	37.16	22.50% (seed 2026)
+ Cutout / Random Erasing	39.0	30.93	28.57% (seed 2026)
+ Mixup / CutMix + label smoothing	7.0	9.83	10.86% (seed 2026)
Final model	None	None	None

In [151…

```
tf.keras.utils.set_random_seed(SEED)

model_baseline = build_model(augmentation=None)

history_baseline = model_baseline.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 165ms/step - accuracy: 0.0825 - loss: 9.7766 - val_acc
uracy: 0.1300 - val_loss: 2.4498
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.1275 - loss: 2.3932 - val_accu
racy: 0.1100 - val_loss: 2.2726
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.1500 - loss: 2.2827 - val_accu
racy: 0.1400 - val_loss: 2.2595
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.1225 - loss: 2.2702 - val_accu
racy: 0.1300 - val_loss: 2.2411
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.1150 - loss: 2.2591 - val_accu
racy: 0.1400 - val_loss: 2.1946
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.1350 - loss: 2.2638 - val_accu
racy: 0.1600 - val_loss: 2.2151
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.1325 - loss: 2.2284 - val_accu
racy: 0.1800 - val_loss: 2.1460
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.1600 - loss: 2.2079 - val_accu
racy: 0.2100 - val_loss: 2.1328
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.1600 - loss: 2.2005 - val_accu
racy: 0.2200 - val_loss: 2.1138
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.2125 - loss: 2.1621 - val_accu
racy: 0.1500 - val_loss: 2.0663
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.1900 - loss: 2.1418 - val_accu
racy: 0.2000 - val_loss: 2.0403
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.1975 - loss: 2.1193 - val_accu
racy: 0.2200 - val_loss: 2.0420
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.2175 - loss: 2.1117 - val_accu
racy: 0.2800 - val_loss: 2.0108
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.2125 - loss: 2.0766 - val_accu
racy: 0.2600 - val_loss: 2.0031
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.2175 - loss: 2.0494 - val_accu
racy: 0.2800 - val_loss: 2.0065
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.2375 - loss: 2.0208 - val_accu
racy: 0.3300 - val_loss: 1.8671
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.2775 - loss: 1.9649 - val_accu
racy: 0.2100 - val_loss: 1.9279
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2925 - loss: 1.9007 - val_accu
racy: 0.2900 - val_loss: 1.9052
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.2625 - loss: 1.9600 - val_accu
```

```
racy: 0.3300 - val_loss: 1.9105
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2875 - loss: 1.8955 - val_accu
racy: 0.3200 - val_loss: 1.8407
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2825 - loss: 1.8638 - val_accu
racy: 0.3600 - val_loss: 1.8013
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.3425 - loss: 1.7923 - val_accu
racy: 0.3300 - val_loss: 1.7724
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.2850 - loss: 1.7867 - val_accu
racy: 0.2700 - val_loss: 1.8667
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.3325 - loss: 1.7251 - val_accu
racy: 0.3700 - val_loss: 1.8260
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.3900 - loss: 1.6572 - val_accu
racy: 0.3000 - val_loss: 1.7894
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.3600 - loss: 1.7248 - val_accu
racy: 0.2900 - val_loss: 1.7738
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.3525 - loss: 1.6897 - val_accu
racy: 0.3200 - val_loss: 1.7726
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.4000 - loss: 1.6330 - val_accu
racy: 0.4000 - val_loss: 1.7237
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.4275 - loss: 1.5571 - val_accu
racy: 0.3600 - val_loss: 1.7287
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.4275 - loss: 1.5265 - val_accu
racy: 0.4200 - val_loss: 1.6811
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.4325 - loss: 1.5226 - val_accu
racy: 0.3600 - val_loss: 1.8495
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.4450 - loss: 1.4710 - val_accu
racy: 0.2800 - val_loss: 1.7252
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.4450 - loss: 1.4813 - val_accu
racy: 0.4200 - val_loss: 1.7288
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.4950 - loss: 1.4072 - val_accu
racy: 0.4300 - val_loss: 1.6629
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 0.5100 - loss: 1.3771 - val_accu
racy: 0.4000 - val_loss: 1.6725
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.4850 - loss: 1.3824 - val_accu
racy: 0.3000 - val_loss: 1.7113
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.5000 - loss: 1.2718 - val_accu
racy: 0.3400 - val_loss: 1.7022
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.4975 - loss: 1.2967 - val_accu
racy: 0.3500 - val_loss: 1.7420
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.5025 - loss: 1.2897 - val_accu
racy: 0.3600 - val_loss: 1.6778
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.5525 - loss: 1.1955 - val_accu
racy: 0.3400 - val_loss: 1.7239
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.5650 - loss: 1.2011 - val_accu
racy: 0.3700 - val_loss: 1.6464
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.5650 - loss: 1.2233 - val_accu
racy: 0.3600 - val_loss: 1.7745
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.5650 - loss: 1.2479 - val_accu
racy: 0.3900 - val_loss: 1.7615
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.5750 - loss: 1.1422 - val_accu
racy: 0.3000 - val_loss: 1.8339
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.6250 - loss: 1.1020 - val_accu
racy: 0.3400 - val_loss: 1.7803
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 58ms/step - accuracy: 0.6050 - loss: 1.0740 - val_accu
racy: 0.3300 - val_loss: 1.7715
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.6350 - loss: 1.0307 - val_accu
racy: 0.3500 - val_loss: 1.6870
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.6050 - loss: 1.0399 - val_accu
racy: 0.3400 - val_loss: 1.9254
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.6150 - loss: 1.0327 - val_accu
racy: 0.3500 - val_loss: 1.7637
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.6575 - loss: 0.9426 - val_accu
racy: 0.3000 - val_loss: 1.9870
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.6475 - loss: 0.9602 - val_accu
racy: 0.3100 - val_loss: 1.8870
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.6275 - loss: 0.9963 - val_accu
racy: 0.3100 - val_loss: 1.8111
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.6550 - loss: 0.9667 - val_accu
racy: 0.3200 - val_loss: 1.9290
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.6450 - loss: 0.9337 - val_accu
racy: 0.3900 - val_loss: 1.9614
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.6575 - loss: 0.9268 - val_accu
racy: 0.3600 - val_loss: 1.9005
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - accuracy: 0.6850 - loss: 0.8681 - val_accu
racy: 0.3200 - val_loss: 1.9698
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.6525 - loss: 0.9257 - val_accu
racy: 0.4100 - val_loss: 1.9105
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.6675 - loss: 0.8900 - val_accu
racy: 0.3300 - val_loss: 1.9095
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.6975 - loss: 0.8468 - val_accu
racy: 0.4100 - val_loss: 1.8817
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.7225 - loss: 0.7668 - val_accu
racy: 0.3100 - val_loss: 2.0448
```

In [152…

```python
_, val_acc_base = model_baseline.evaluate(
    x_val, y_val, verbose=0
)

_, clean_acc_base = model_baseline.evaluate(
    x_clean, y_clean, verbose=0
)

_, shifted_acc_base = model_baseline.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Validation Accuracy:", round(val_acc_base * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc_base * 100, 2), "%")
print("Synthetic Shifted Accuracy:", round(shifted_acc_base * 100, 2), "%")
```

```
Validation Accuracy: 31.0 %
Clean Test Accuracy: 35.49 %
Synthetic Shifted Accuracy: 19.48 %
```

In [153…

```python
df_results.loc[0, "Val Acc"] = round(
    val_acc_base * 100, 2
)

df_results.loc[0, "Clean Test"] = round(
    clean_acc_base * 100, 2
)

df_results.loc[0, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc_base * 100:.2f}% (seed 2026)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	31.0	35.49	19.48% (seed 2026)
Geometric only	38.0	34.38	23.74% (seed 2026)
Photometric only	35.0	36.89	18.83% (seed 2026)
Geometric + photometric	38.0	37.16	22.50% (seed 2026)
+ Cutout / Random Erasing	39.0	30.93	28.57% (seed 2026)
+ Mixup / CutMix + label smoothing	7.0	9.83	10.86% (seed 2026)
Final model	None	None	None

In [154…

```
tf.keras.utils.set_random_seed(SEED)

final_augmentation = keras.Sequential([
    layers.RandomTranslation(
        0.1, 0.1,
        fill_mode="nearest"
    ),

    layers.RandomFlip("horizontal"),

    layers.RandomRotation(
        25 / 360,
        fill_mode="nearest"
    ),

    layers.RandomBrightness(
        factor=0.2,
        value_range=(0, 255)
    ),

    layers.RandomContrast(factor=0.2),

    layers.Lambda(apply_cutout)
])
```

In [155…

```
model_final = build_model(final_augmentation)

history_final = model_final.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 6s 271ms/step - accuracy: 0.0875 - loss: 6.7807 - val_acc
uracy: 0.1000 - val_loss: 2.4071
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.1100 - loss: 2.3455 - val_accu
racy: 0.1100 - val_loss: 2.2584
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.1225 - loss: 2.2893 - val_accu
racy: 0.1400 - val_loss: 2.2582
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.1400 - loss: 2.2725 - val_accu
racy: 0.1600 - val_loss: 2.2405
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.1200 - loss: 2.2591 - val_accu
racy: 0.1200 - val_loss: 2.2187
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.1375 - loss: 2.2332 - val_accu
racy: 0.1400 - val_loss: 2.2058
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.1450 - loss: 2.2688 - val_accu
racy: 0.1400 - val_loss: 2.2378
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.1600 - loss: 2.2218 - val_accu
racy: 0.1500 - val_loss: 2.1986
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.1575 - loss: 2.2066 - val_accu
racy: 0.1700 - val_loss: 2.1537
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 66ms/step - accuracy: 0.1600 - loss: 2.2031 - val_accu
racy: 0.2000 - val_loss: 2.1224
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.1225 - loss: 2.1977 - val_accu
racy: 0.2100 - val_loss: 2.1052
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.1925 - loss: 2.1502 - val_accu
racy: 0.2400 - val_loss: 2.1168
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.1975 - loss: 2.1725 - val_accu
racy: 0.1900 - val_loss: 2.0889
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2025 - loss: 2.1491 - val_accu
racy: 0.2100 - val_loss: 2.0427
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.1750 - loss: 2.1563 - val_accu
racy: 0.2100 - val_loss: 2.0871
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.1750 - loss: 2.1508 - val_accu
racy: 0.2400 - val_loss: 2.0602
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.1925 - loss: 2.1390 - val_accu
racy: 0.1800 - val_loss: 2.0423
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.1625 - loss: 2.1343 - val_accu
racy: 0.2200 - val_loss: 2.0335
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2125 - loss: 2.1253 - val_accu
```

```
racy: 0.2500 - val_loss: 2.0424
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2100 - loss: 2.1134 - val_accu
racy: 0.2400 - val_loss: 2.0070
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - accuracy: 0.1975 - loss: 2.1245 - val_accu
racy: 0.2500 - val_loss: 2.0250
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2000 - loss: 2.0872 - val_accu
racy: 0.2200 - val_loss: 2.0137
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2150 - loss: 2.0678 - val_accu
racy: 0.2700 - val_loss: 2.0096
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 80ms/step - accuracy: 0.1925 - loss: 2.1103 - val_accu
racy: 0.2300 - val_loss: 2.0038
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2450 - loss: 2.0636 - val_accu
racy: 0.2300 - val_loss: 1.9906
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2075 - loss: 2.0804 - val_accu
racy: 0.2400 - val_loss: 1.9965
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2300 - loss: 2.0690 - val_accu
racy: 0.2000 - val_loss: 1.9547
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2350 - loss: 2.0059 - val_accu
racy: 0.3200 - val_loss: 1.9307
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2850 - loss: 1.9920 - val_accu
racy: 0.2900 - val_loss: 1.9000
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - accuracy: 0.2225 - loss: 2.0335 - val_accu
racy: 0.2500 - val_loss: 1.9381
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.2275 - loss: 2.0226 - val_accu
racy: 0.2900 - val_loss: 1.9210
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step - accuracy: 0.2225 - loss: 2.0233 - val_accu
racy: 0.3100 - val_loss: 1.9256
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2500 - loss: 1.9978 - val_accu
racy: 0.2900 - val_loss: 1.8805
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2825 - loss: 1.9430 - val_accu
racy: 0.3000 - val_loss: 1.9123
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2800 - loss: 1.9563 - val_accu
racy: 0.2300 - val_loss: 1.8881
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2700 - loss: 1.9460 - val_accu
racy: 0.2700 - val_loss: 1.8388
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2200 - loss: 1.9894 - val_accu
racy: 0.3700 - val_loss: 1.8596
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2900 - loss: 1.9638 - val_accu
racy: 0.3000 - val_loss: 1.9032
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.2575 - loss: 1.9492 - val_accu
racy: 0.2700 - val_loss: 1.8944
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.2150 - loss: 1.9896 - val_accu
racy: 0.2900 - val_loss: 1.8711
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.2675 - loss: 1.9406 - val_accu
racy: 0.2600 - val_loss: 1.9017
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2625 - loss: 1.9701 - val_accu
racy: 0.3000 - val_loss: 1.8851
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 61ms/step - accuracy: 0.2725 - loss: 1.9583 - val_accu
racy: 0.2700 - val_loss: 1.9294
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - accuracy: 0.2750 - loss: 1.9214 - val_accu
racy: 0.1800 - val_loss: 1.9100
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.2775 - loss: 1.9446 - val_accu
racy: 0.3200 - val_loss: 1.9016
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.3200 - loss: 1.9067 - val_accu
racy: 0.3300 - val_loss: 1.8356
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2625 - loss: 1.9143 - val_accu
racy: 0.3100 - val_loss: 1.8129
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 76ms/step - accuracy: 0.2625 - loss: 1.8789 - val_accu
racy: 0.2500 - val_loss: 1.8687
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2825 - loss: 1.9224 - val_accu
racy: 0.3100 - val_loss: 1.9062
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2950 - loss: 1.8957 - val_accu
racy: 0.3000 - val_loss: 1.8837
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2975 - loss: 1.8983 - val_accu
racy: 0.3200 - val_loss: 1.8907
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.2550 - loss: 1.9464 - val_accu
racy: 0.3100 - val_loss: 1.8412
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.3350 - loss: 1.8523 - val_accu
racy: 0.3300 - val_loss: 1.8228
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2900 - loss: 1.8647 - val_accu
racy: 0.3500 - val_loss: 1.8011
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.3150 - loss: 1.8542 - val_accu
racy: 0.3800 - val_loss: 1.8286
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.3350 - loss: 1.7982 - val_accu
racy: 0.2700 - val_loss: 1.8239
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - accuracy: 0.3325 - loss: 1.8559 - val_accu
racy: 0.3600 - val_loss: 1.8055
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.3475 - loss: 1.8361 - val_accu
racy: 0.3400 - val_loss: 1.8123
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.2875 - loss: 1.8599 - val_accu
racy: 0.3100 - val_loss: 1.8467
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.2800 - loss: 1.8556 - val_accu
racy: 0.3400 - val_loss: 1.7602
```

In [156…

```python
_, final_val_acc = model_final.evaluate(
    x_val, y_val, verbose=0
)

_, final_clean_acc = model_final.evaluate(
    x_clean, y_clean, verbose=0
)

_, final_shifted_acc = model_final.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Final Validation Accuracy:",
      round(final_val_acc * 100, 2), "%")

print("Final Clean Test Accuracy:",
      round(final_clean_acc * 100, 2), "%")

print("Final Synthetic Shifted Accuracy:",
      round(final_shifted_acc * 100, 2), "%")
```

```
Final Validation Accuracy: 31.0 %
Final Clean Test Accuracy: 29.96 %
Final Synthetic Shifted Accuracy: 27.9 %
```

In [157…

```python
df_results.loc[6, "Val Acc"] = round(
    final_val_acc * 100, 2
)

df_results.loc[6, "Clean Test"] = round(
    final_clean_acc * 100, 2
)

df_results.loc[6, "Shifted Test (Mean ± Std)"] = (
    f"{final_shifted_acc * 100:.2f}% (seed 2026)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	31.0	35.49	19.48% (seed 2026)
Geometric only	38.0	34.38	23.74% (seed 2026)
Photometric only	35.0	36.89	18.83% (seed 2026)
Geometric + photometric	38.0	37.16	22.50% (seed 2026)
+ Cutout / Random Erasing	39.0	30.93	28.57% (seed 2026)
+ Mixup / CutMix + label smoothing	7.0	9.83	10.86% (seed 2026)
Final model	31.0	29.96	27.90% (seed 2026)

In [158…
```
model_final.save("cifar10_final_model.keras")
```

<h3 id="PART-E">PART E</h3>

In [160…
```
y_prob = model_final.predict(x_clean, verbose=1)

y_pred = np.argmax(y_prob, axis=1)
y_true = y_clean.reshape(-1)

print("Clean Test Accuracy:",
      np.mean(y_pred == y_true) * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step
Clean Test Accuracy: 30.330000000000002 %
```

In [161…
```
class_results = []

for i, name in enumerate(class_names):
    mask = (y_true == i)
    correct = np.sum(y_pred[mask] == y_true[mask])
    total = np.sum(mask)

    class_results.append({
        "Class": name,
        "Correct": correct,
        "Total": total,
        "Accuracy (%)": round(correct / total * 100, 2)
    })

df_class_accuracy = pd.DataFrame(class_results)

display(df_class_accuracy)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Class</th> <th>Correct</th> <th>Total</th> <th>Accuracy (%)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

airplane	124	1000	12.4
automobile	572	1000	57.2
bird	74	1000	7.4
cat	96	1000	9.6
deer	290	1000	29.0
dog	291	1000	29.1
frog	262	1000	26.2
horse	499	1000	49.9
ship	569	1000	56.9
truck	256	1000	25.6

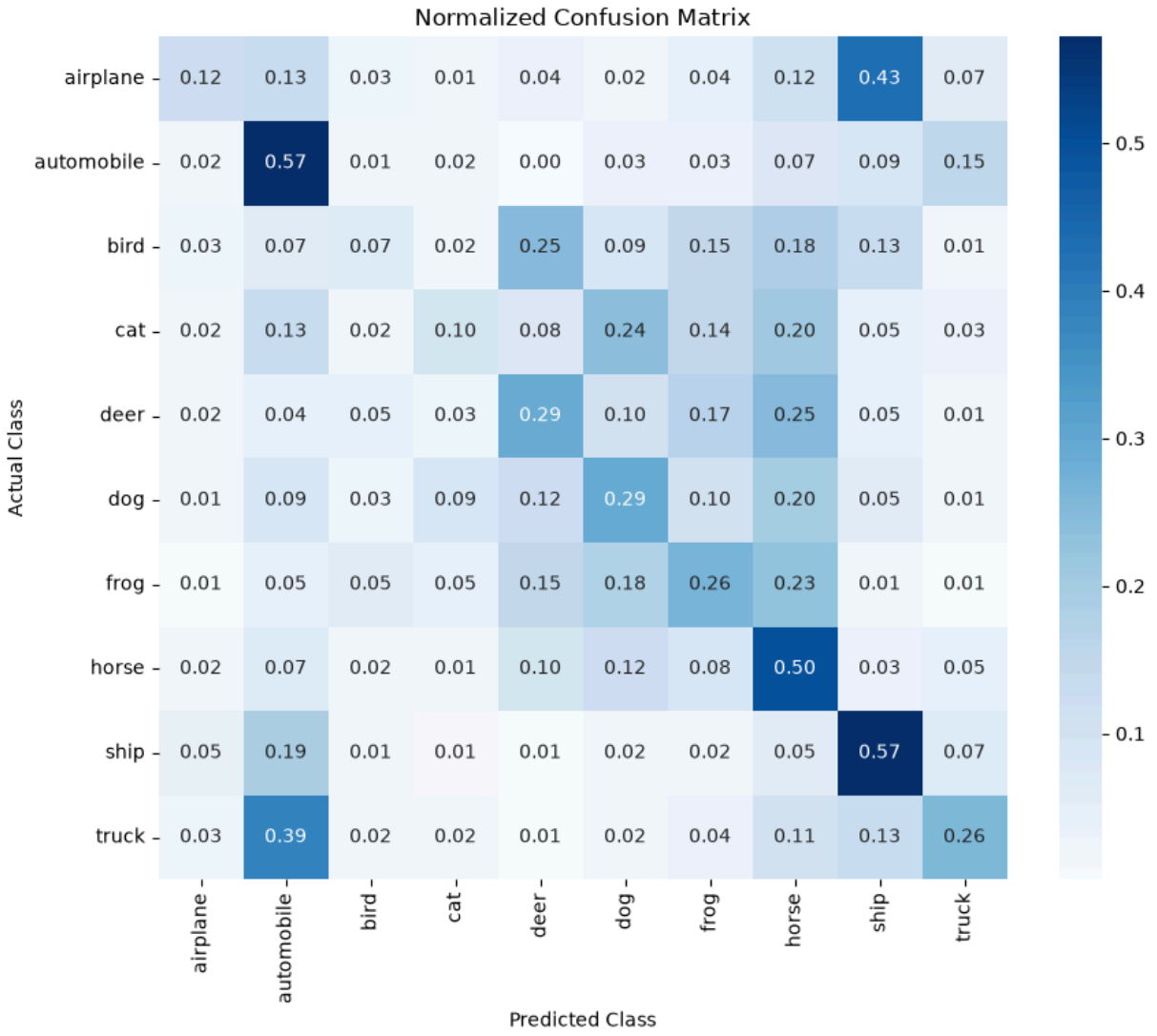
In [168…

```
cm = confusion_matrix(
    y_true,
    y_pred,
    normalize="true"
)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt=".2f",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Normalized Confusion Matrix")
plt.show()
```



In [169…

```
cm_counts = confusion_matrix(y_true, y_pred)
np.fill_diagonal(cm_counts, 0)

pairs = []

for i in range(10):
    for j in range(10):
        if i != j:
            pairs.append((
                cm_counts[i, j],
                class_names[i],
                class_names[j]
            ))

pairs.sort(reverse=True)

for count, actual, predicted in pairs[:5]:
    print(f"Actual: {actual} -> Predicted: {predicted}, Count: {count}")
```

```
Actual: airplane -> Predicted: ship, Count: 432
Actual: truck -> Predicted: automobile, Count: 386
Actual: deer -> Predicted: horse, Count: 247
Actual: bird -> Predicted: deer, Count: 247
Actual: cat -> Predicted: dog, Count: 241
```

In [170…

```python
wrong_indices = np.where(y_pred != y_true)[0]

if len(wrong_indices) >= 8:
    plt.figure(figsize=(12, 7))

    for k, idx in enumerate(wrong_indices[:8]):
        plt.subplot(2, 4, k + 1)

        plt.imshow(
            np.clip(x_clean[idx], 0, 255).astype("uint8")
        )

        plt.title(
            f"True: {class_names[y_true[idx]]}\n"
            f"Pred: {class_names[y_pred[idx]]}"
        )

        plt.axis("off")

    plt.tight_layout()
    plt.show()
else:
    print("Fewer than 8 misclassified images found.")
```



In [174…

```python
x_hidden = tf.image.rgb_to_grayscale(
    tf.cast(x_clean, tf.float32)
)

x_hidden = tf.image.grayscale_to_rgb(x_hidden)
```

```
x_hidden = tf.cast(x_hidden, x_clean.dtype)

print("Hidden images shape:", x_hidden.shape)
print("Clean images shape:", x_clean.shape)
```

```
Hidden images shape: (10000, 32, 32, 3)
Clean images shape: (10000, 32, 32, 3)
```

In [172…

```
print("Clean test shape:", x_clean.shape)
print("Clean labels shape:", y_clean.shape)
```

```
Clean test shape: (10000, 32, 32, 3)
Clean labels shape: (10000,)
```

In [175…

```
hidden_loss, hidden_acc = model_final.evaluate(
    x_hidden,
    y_clean,
    verbose=1
)

print(
    "Hidden Corruption Accuracy:",
    round(hidden_acc * 100, 2),
    "%"
)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.1874 - loss: 2.1746
Hidden Corruption Accuracy: 18.74 %
```

The class accuracy table shows how well the model predicts each class. The confusion matrix shows which classes the model confuses most. We check at least 8 wrong images to find the reasons for the mistakes, such as similar shapes or unclear images. Finally, we test the model on hidden corruption to see if it can handle new changes in images.

In [ ]: